Học viện Công nghệ Bưu chính Viễn thông
Khoa Công nghệ Thông tin

Intelligent System Development — Tiểu luận Final

Phát triển hệ thống thông minh

từ học máy đến hệ đa agent cho bệnh đái tháo đường

Nhóm 02

Trần Xuân Thành — B23DCAT280 — E23CNPM01

Giảng viên hướng dẫn: Dinh Que Tran, Ph.D., Assoc. Prof.

Học kỳ I.2026

Mục lục

1 Học máy **4**
- 1.1 Học có giám sát . . . 4
  - 1.1.1 Đặt bài toán . . . 4
  - 1.1.2 Vòng đời một hệ học máy . . . 4
- 1.2 Dữ liệu . . . 4
  - 1.2.1 Biểu diễn . . . 4
  - 1.2.2 Chia tập và rò rỉ dữ liệu . . . 5
- 1.3 Các mô hình cổ điển . . . 5
- 1.4 Đánh giá . . . 5
  - 1.4.1 Hồi quy . . . 5
  - 1.4.2 Phân loại . . . 5
- 1.5 Demo các assignment . . . 6
  - 1.5.1 Assignment 1: dự đoán giá nhà Việt Nam 2024 . . . 6
  - 1.5.2 Assignment 2: ba ứng dụng trên một đường ống chung . . . 7
  - 1.5.3 Assignment 3: mạng nơ-ron 3 lớp viết bằng NumPy . . . 8

2 Mạng nơ-ron tích chập **10**
- 2.1 Mạng nơ-ron nhiều lớp . . . 10
  - 2.1.1 Mạng là hợp của các hàm sơ cấp . . . 10
  - 2.1.2 Lan truyền ngược . . . 10
  - 2.1.3 Tối ưu và điều chuẩn . . . 10
- 2.2 Phép tích chập . . . 10
  - 2.2.1 Định nghĩa . . . 10
  - 2.2.2 Kích thước, pooling, vùng tiếp nhận . . . 11
  - 2.2.3 Cài đặt bằng im2col . . . 11
  - 2.2.4 Một kiến trúc CNN điển hình . . . 12
- 2.3 Demo các assignment . . . 12
  - 2.3.1 Assignment 4: một kiến trúc, ba bản cài đặt . . . 12
  - 2.3.2 Assignment 5: Keras và PyTorch, 3 tầng và 5 tầng . . . 13
  - 2.3.3 CNN trên dữ liệu bảng trong project . . . 14

3 Mạng nơ-ron hồi quy **16**
- 3.1 Dữ liệu chuỗi . . . 16
- 3.2 Cấu trúc RNN . . . 16
- 3.3 Huấn luyện . . . 16
  - 3.3.1 Lan truyền ngược theo thời gian . . . 16
  - 3.3.2 Gradient tắt dần và bùng nổ . . . 17
  - 3.3.3 Cài đặt . . . 17
- 3.4 Demo các assignment . . . 17
  - 3.4.1 Assignment 6: dự báo số giao dịch theo giờ . . . 17
  - 3.4.2 Assignment 7, phần RNN: RNN là hợp của các hàm sơ cấp . . . 18

4 Long Short-Term Memory **19**
- 4.1 Ô nhớ và các cổng . . . 19
- 4.2 Biến thể và chi phí . . . 20
- 4.3 Demo các assignment . . . 20
  - 4.3.1 Assignment 7: so sánh RNN và LSTM . . . 20

4.3.2 LSTM trên dữ liệu bảng trong project 21

5 Knowledge Base và RAG **23**
5.1 Vì sao cần KB và RAG 23
5.1.1 Giới hạn của mô hình ngôn ngữ lớn 23
5.1.2 Hai cách bổ sung tri thức 23
5.2 Knowledge Base 24
5.2.1 Khái niệm 24
5.2.2 Vì sao chọn Neo4j 24
5.3 Xây KB từ link và file PDF 24
5.3.1 Nguồn tài liệu 24
5.3.2 Hai luồng thu thập 25
5.4 Biểu diễn tri thức 27
5.4.1 Vector của đoạn văn 27
5.4.2 Lược đồ đồ thị 27
5.4.3 Trích dữ kiện y tế có kiểm chứng 28
5.5 Pipeline RAG 28
5.5.1 Truy xuất 28
5.5.2 Sinh câu trả lời 29
5.5.3 Cổng trích dẫn 29
5.5.4 Phục vụ 30
5.6 Đánh giá 31
5.6.1 Phương pháp 31
5.6.2 So sánh các cách truy xuất 32
5.6.3 Đóng góp của đồ thị 32
5.6.4 Chất lượng câu trả lời 33
5.7 Hạn chế và vai trò trong hệ đa agent 33

6 Agent và hệ đa agent **35**
6.1 Agent 35
6.1.1 Định nghĩa 35
6.1.2 Các loại agent cổ điển 35
6.1.3 Agent dựa trên mô hình ngôn ngữ lớn 36
6.1.4 Vòng lặp ReAct 36
6.1.5 Workflow và agent 37
6.2 Hệ đa agent 37
6.2.1 Khái niệm 37
6.2.2 Vì sao chia thành nhiều agent 37
6.2.3 Các mẫu kiến trúc 38
6.2.4 Giao tiếp và trạng thái chung 38
6.2.5 Framework 38
6.3 Diabeque: hệ đa agent cho đái tháo đường 39
6.3.1 Phạm vi: sàng lọc, không chẩn đoán 39
6.3.2 Các agent 39
6.3.3 Kiến trúc 42
6.3.4 Trạng thái chung và cài đặt bằng LangGraph 42
6.3.5 Giao diện 43
6.3.6 Kết quả đánh giá 43
6.4 Hạn chế và rủi ro 44

Tài liệu tham khảo **46**

Chương 1

Học máy

Chương này trình bày nền tảng học máy có giám sát dùng xuyên suốt tiểu luận: cách đặt bài toán, biểu diễn và chia dữ liệu, các mô hình cổ điển, và cách đánh giá. Phần cuối tóm tắt các demo của Assignment 1–3.

1.1. Học có giám sát

1.1.1 Đặt bài toán

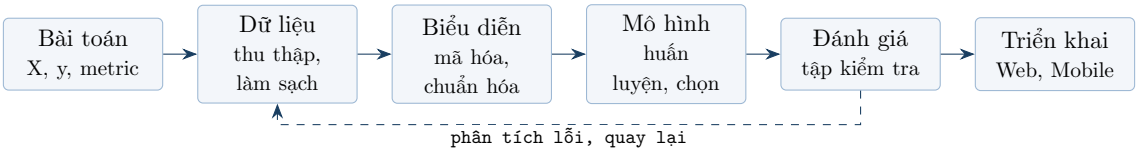
Cho tập dữ liệu $\mathcal{D} = \{(x_i, y_i)\}_{i=1}^{N}$, với $x_i \in \mathbb{R}^d$ là vector đặc trưng và y_i là nhãn. Học có giám sát tìm hàm f_θ sao cho $f_\theta(x) \approx y$ trên dữ liệu *chưa thấy*. Khi y là số thực, bài toán là **hồi quy** (ví dụ giá nhà); khi y thuộc một tập hữu hạn lớp, bài toán là **phân loại** (ví dụ nhóm tình trạng đái tháo đường).

Tham số được chọn bằng cực tiểu rủi ro thực nghiệm, có thêm số hạng điều chuẩn:

$$\hat{\theta} = \arg\min_{\theta} \frac{1}{N} \sum_{i=1}^{N} \ell(f_\theta(x_i), y_i) + \lambda\, \Omega(\theta).$$

Hàm mất mát ℓ đo sai lệch trên một mẫu; Ω phạt mô hình phức tạp (ví dụ $\|\theta\|_2^2$ với ridge, $\|\theta\|_1$ với lasso).

1.1.2 Vòng đời một hệ học máy



Hình 1.1: Vòng đời một hệ học máy.

Metric phải được chọn từ đầu, theo mục tiêu thực tế, không theo mô hình: với sàng lọc bệnh, bỏ sót người bệnh (recall thấp) thường đắt hơn báo động nhầm.

1.2. Dữ liệu

1.2.1 Biểu diễn

Mô hình chỉ nhận số, nên mỗi loại dữ liệu cần một cách biểu diễn:

- **Biến liên tục** (BMI, diện tích): chuẩn hóa $z = (x - \mu)/\sigma$ để các cột có cùng thang đo.
- **Biến thứ bậc** (nhóm tuổi, mức sức khỏe 1–5): giữ số thứ tự.
- **Biến định danh** (quận, loại nhà): one-hot, mỗi giá trị một cột 0/1; không mã thành 1, 2, 3 vì sẽ tạo thứ tự giả.
- **Văn bản**: túi từ, TF-IDF, hoặc embedding (Chương 3).
- **Ảnh**: tensor cao × rộng × kênh (Chương 2).

Cột được phân loại theo bản chất, không theo kiểu lưu trữ: mã bưu điện lưu bằng số nhưng là biến định danh.

1.2.2 Chia tập và rò rỉ dữ liệu

Dữ liệu được chia thành ba tập: *huấn luyện* để học tham số, *xác thực* để chọn mô hình và siêu tham số, *kiểm tra* chỉ dùng một lần để báo cáo. Rò rỉ dữ liệu xảy ra khi thông tin của tập kiểm tra lọt vào bước học, làm điểm số đẹp hơn thực tế. Các dạng thường gặp:

1. tính trung bình, độ lệch chuẩn, trung vị điền thiếu trên toàn bộ dữ liệu thay vì chỉ trên tập huấn luyện;
2. dòng trùng lặp nằm ở cả tập huấn luyện và tập kiểm tra (BRFSS: 686.019 dòng nhưng chỉ 634.636 hồ sơ khác nhau, nên phải chia theo hồ sơ);
3. đặc trưng chứa đáp án (câu hỏi "đang dùng insulin" gần như trả lời thẳng Type 1 hay Type 2);
4. chia ngẫu nhiên dữ liệu theo thời gian, để mô hình "nhìn thấy tương lai" (Chương 3).

1.3. Các mô hình cổ điển

Hồi quy tuyến tính. $\hat{y} = w^\top x + b$, cực tiểu sai số bình phương trung bình $\mathrm{MSE} = \frac{1}{N}\sum_i (y_i - \hat{y}_i)^2$; có nghiệm đóng $w = (X^\top X)^{-1} X^\top y$.

Hồi quy logistic. Cho phân loại nhị phân: $p(y{=}1 \mid x) = \sigma(w^\top x + b)$ với $\sigma(z) = 1/(1+e^{-z})$; hàm mất mát cross-entropy

$$\ell = -[y \log p + (1-y)\log(1-p)].$$

Với K lớp, thay σ bằng softmax $p_k = e^{z_k} / \sum_j e^{z_j}$. Hồi quy logistic là một mạng nơ-ron không có lớp ẩn, nên là mốc so sánh tự nhiên cho các mạng ở các chương sau.

Cây quyết định và mô hình tổ hợp. Cây chia không gian đặc trưng bằng các câu hỏi "$x_j \le t$?", mỗi lần chọn phép chia làm giảm độ hỗn tạp nhiều nhất (Gini $1 - \sum_k p_k^2$ hoặc entropy). Một cây sâu dễ quá khớp. *Random forest* lấy trung bình nhiều cây học trên các mẫu bootstrap và tập con đặc trưng ngẫu nhiên, giảm phương sai. *Gradient boosting* thêm lần lượt các cây nhỏ, mỗi cây học phần sai số còn lại của các cây trước, giảm độ chệch.

kNN và SVM. kNN dự đoán theo đa số của k điểm gần nhất, không có bước huấn luyện. SVM tìm siêu phẳng có lề lớn nhất giữa hai lớp; hàm nhân (kernel) cho phép biên phi tuyến.

Quá khớp và đánh đổi độ chệch–phương sai. Mô hình quá đơn giản không bắt được quy luật (chệch cao); mô hình quá phức tạp học cả nhiễu của tập huấn luyện (phương sai cao). Dấu hiệu quá khớp: sai số huấn luyện tiếp tục giảm trong khi sai số xác thực đi ngang hoặc tăng. Cách xử lý: điều chuẩn, thêm dữ liệu, dừng sớm, giảm độ phức tạp.

1.4. Đánh giá

1.4.1 Hồi quy

$\mathrm{MAE} = \frac{1}{N}\sum |y_i - \hat{y}_i|$ (cùng đơn vị với y, ít nhạy với ngoại lai), $\mathrm{RMSE} = \sqrt{\mathrm{MSE}}$ (phạt nặng sai số lớn), và $R^2 = 1 - \sum (y_i - \hat{y}_i)^2 / \sum (y_i - \bar{y})^2$ (tỷ lệ phương sai được giải thích).

1.4.2 Phân loại

Từ ma trận nhầm lẫn, với một lớp: precision $= TP/(TP+FP)$, recall $= TP/(TP+FN)$, $F_1 = 2PR/(P+R)$. Macro-F1 lấy trung bình F_1 các lớp với trọng số bằng nhau. ROC-AUC

đo khả năng xếp hạng: xác suất một mẫu dương ngẫu nhiên được chấm điểm cao hơn một mẫu âm ngẫu nhiên.

Lớp lệch. Khi một lớp chiếm 84% dữ liệu (như BRFSS), mô hình luôn đoán lớp đó đạt accuracy 0,84 mà không tìm ra người bệnh nào. Ba công cụ thường dùng: chọn mô hình bằng macro-F1 thay vì accuracy; trọng số lớp $w_c \propto 1/n_c$ trong hàm mất mát; chọn ngưỡng quyết định trên tập xác thực thay vì cố định 0,5.

Hiệu chỉnh xác suất. Một mô hình được hiệu chỉnh tốt nếu trong các dự đoán có độ tự tin 0,7 thì khoảng 70% là đúng. ECE (expected calibration error) chia dự đoán theo độ tự tin thành các khoảng và lấy trung bình có trọng số của độ chênh giữa độ tự tin và tỷ lệ đúng. Temperature scaling chia logit cho một hằng số T chọn trên tập xác thực: $p = \text{softmax}(z/T)$; cách này không đổi nhãn dự đoán, chỉ đổi độ tự tin.

1.5. Demo các assignment

1.5.1 Assignment 1: dự đoán giá nhà Việt Nam 2024

Bài toán. Hồi quy giá nhà (tỷ VNĐ) từ 12 thuộc tính của Vietnam Housing Dataset 2024; chia 80/20 thành 2.000 mẫu huấn luyện và 500 mẫu kiểm tra. Mô hình tốt nhất được đóng gói vào một ứng dụng web giao diện điện thoại, chạy dự đoán ngay trên máy người dùng.

Bảng 1.1: A1: kết quả trên tập kiểm tra

Mô hình	MAE (tỷ)	RMSE (tỷ)	R^2	MAPE
Mốc: trung vị	4,4715	6,7256	−0,0536	46,39%
Ridge	1,7389	2,5073	0,8536	19,54%
Cây quyết định	1,7187	2,9059	0,8033	16,16%
Random forest	1,3522	2,0892	0,8983	12,80%
Gradient boosting	**1,2013**	**1,9211**	**0,9140**	**11,14%**

Điểm chính.

- Mô hình tổ hợp cây thắng rõ: gradient boosting giảm MAE còn khoảng một phần tư so với mốc trung vị.
- Chuẩn hóa là điều kiện để Ridge hoạt động: diện tích và số tầng có khoảng giá trị chênh nhau rất xa.
- Tăng số cây của rừng từ 10 lên 100 nâng R^2 từ 0,72 lên 0,9140; quá 200 cây không cải thiện thêm.

Đoạn mã dưới đây cho thấy cách A1 tránh rò rỉ dữ liệu: bộ chuẩn hóa và bộ mã one-hot nằm trong cùng `Pipeline` với mô hình, nên `fit` chỉ học trung bình, độ lệch chuẩn và danh sách giá trị định danh trên `X_train`. Năm mô hình dùng chung một bộ tiền xử lý.

Đoạn mã 1.1. Tiền xử lý theo loại cột và đường ống huấn luyện của A1. `ass_01/prepare_data_and_models.py`, dòng 182–211.

```
X_train, X_test, y_train, y_test = train_test_split(X, y, test_size=0.2, random_state=42)

num_cols = ['Area', 'Frontage', 'Access Road', 'Floors', 'Bedrooms', 'Bathrooms']
cat_cols = ['House direction', 'Balcony direction', 'Legal status', 'Furniture state', 'District']

preprocessor = ColumnTransformer(
    transformers=[
```

```
        ('num', StandardScaler(), num_cols),
        ('cat', OneHotEncoder(handle_unknown='ignore', sparse_output=False), cat_cols)
    ]
)
# ...
for name, model in models.items():
    pipe = Pipeline([
        ('prep', preprocessor),
        ('model', model)
    ])
    pipe.fit(X_train, y_train)
    preds = pipe.predict(X_test)
```

1.5.2 Assignment 2: ba ứng dụng trên một đường ống chung

Bài toán. Ba bài toán dùng chung một đường ống tiền xử lý–huấn luyện–triển khai (FastAPI, web, Flutter): phân loại nguy cơ đái tháo đường trên BRFSS 2015 (253.680 dòng, 21 đặc trưng), hồi quy giá nhà Mỹ (2.226.382 dòng) và dự đoán mức hài lòng khách hàng từ ticket hỗ trợ (85.907 dòng). Tổng cộng 17 mô hình được so sánh.

Bảng 1.2: A2: mô hình được chọn cho mỗi ứng dụng (tập kiểm tra)

Ứng dụng	Mô hình	Kết quả
Đái tháo đường	Random forest, trọng số lớp cân bằng	Recall 0,5832; ROC-AUC 0,8094; F1 0,4890
Giá nhà Mỹ	HistGradientBoosting trên log(giá)	MAE 118.275 USD; R^2 (USD) 0,6624
Mức hài lòng	Hồi quy logistic, bảng + TF-IDF	F1-macro 0,7793; ROC-AUC 0,8848

Điểm chính.

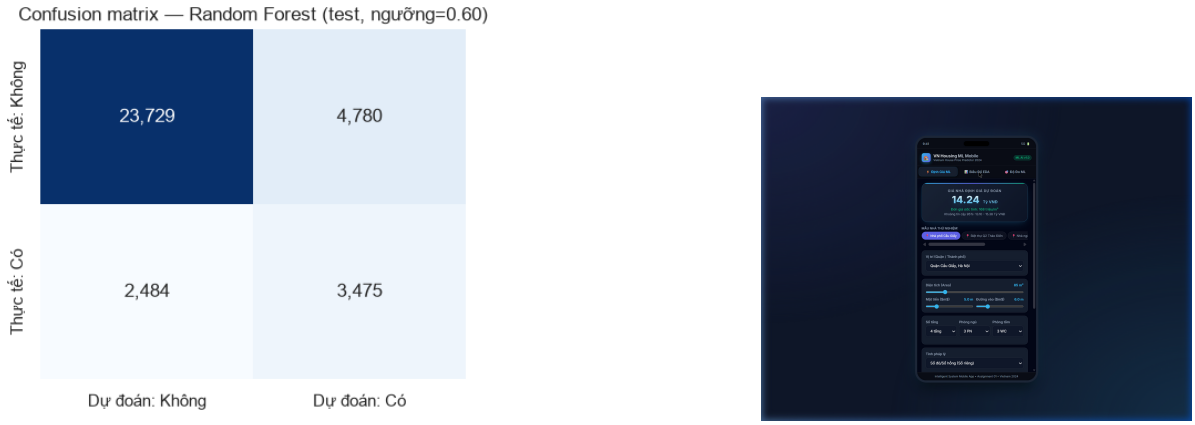
- **Accuracy che giấu thất bại**: HistGradientBoosting ba lớp đạt accuracy 0,8365 nhưng recall lớp tiền đái tháo đường bằng 0. Đây là lý do các chương sau chọn mô hình bằng macro-F1.
- **Báo cáo metric theo đơn vị người dùng quan tâm**: Ridge đạt $R^2 = 0{,}5959$ trên thang log nhưng $-1{,}5688$ khi đổi về USD.
- **Biểu diễn phụ thuộc mục tiêu**: thêm TF-IDF tăng F1-macro 0,1431 cho bài mức hài lòng nhưng giảm 0,0072 cho bài nhóm sản phẩm.
- Mô hình điểm cao nhất chưa chắc nên triển khai: LinearSVC hơn 0,0005 F1-macro nhưng không trả được xác suất.

Bài đái tháo đường của A2 áp dụng trọng số lớp ở mục "Lớp lệch" bằng tham số `class_weight` của scikit-learn. `"balanced"` đặt $w_c = N/(K\,n_c)$; với random forest, `"balanced_subsample"` tính lại trọng số trên từng mẫu bootstrap. Mỗi mô hình vẫn đi kèm bộ tiền xử lý của nó trong một `Pipeline` và chỉ được `fit` trên tập huấn luyện. Mã dưới đây lấy từ bản in notebook `01_diabetes.ipynb` (ô In [23]) trong phụ lục của A2; thụt lề được chuẩn hóa lại.

Đoạn mã 1.2. Trọng số lớp trong các mô hình phân loại của A2. `ass_02/A2_02_TranXuanThanh_280/code/appD_06.txt`, dòng 23–64.

```
def build_models():
    return [
        (
            "Logistic Regression",
            "onehot",
            LogisticRegression(
```

```
                max_iter=1000, class_weight="balanced", random_state=common.RANDOM_SEED
            ),
        ),
        # ...
        (
            "Random Forest",
            "native",
            RandomForestClassifier(
                n_estimators=200, max_depth=14, min_samples_leaf=20,
                class_weight="balanced_subsample", n_jobs=-1, random_state=common.RANDOM_SEED,
            ),
        ),
        # ...
    ]
# ...
for name, rep, est in build_models():
    pipe = Pipeline([("prep", make_preprocessor(rep)), ("model", est)])
    pipe.fit(X_tr, y_tr) # fit DUY NHẤT ở đây trên tập train
```



Hình 1.2: Ma trận nhầm lẫn của mô hình đái tháo đường A2 (trái) và ứng dụng dự đoán giá nhà A1 (phải).

1.5.3 Assignment 3: mạng nơ-ron 3 lớp viết bằng NumPy

Nội dung. Bài giảng 03 là cầu nối từ học máy cổ điển sang học sâu. Ý chính: học máy cổ điển đi theo luồng *dữ liệu → đặc trưng do người thiết kế → mô hình*, còn học sâu để mô hình tự học các biểu diễn trung gian, và một mạng sâu là *hợp của các hàm có tham số*:

$$\hat{y} = (f_3 \circ f_2 \circ f_1)(X), \qquad f_1(X) = \mathrm{ReLU}(XW_1 + b_1).$$

Bài thực hành xây mạng $8 \to 16 \to 8 \to 1$ (ReLU, ReLU, sigmoid) chỉ bằng NumPy, không dùng TensorFlow, PyTorch hay Keras, cho bài phân loại nhị phân đái tháo đường trên bộ Pima (8 chỉ số lâm sàng: số lần mang thai, glucose, huyết áp, độ dày nếp da, insulin, BMI, hệ số di truyền, tuổi). Các bước tự viết: lan truyền tiến, hàm mất mát binary cross-entropy, lan truyền ngược bằng quy tắc dây chuyền, cập nhật gradient descent với $\eta = 0{,}01$, rồi đánh giá bằng accuracy, precision, recall, F1.

Các câu hỏi thí nghiệm. Đề yêu cầu đổi kiến trúc ($8 \to 32 \to 16 \to 1$, bỏ lớp thứ hai), đổi tốc độ học (0,001; 0,01; 0,1), bỏ ReLU, và đổi ngưỡng quyết định (0,3; 0,5; 0,7). Mỗi câu ứng với một ý lý thuyết ở trên: mạng sâu hơn không chắc tốt hơn; tốc độ học quá nhỏ thì hội tụ chậm, quá lớn thì dao động; bỏ hàm kích hoạt thì hợp các hàm tuyến tính vẫn là tuyến tính; ngưỡng thấp tăng recall và giảm precision.

Phần thực hành được làm ở đâu. A3 không nộp riêng: theo đề trên lớp, Assignment 4 bao trùm A3. Mạng MLP viết tay bằng NumPy trên BRFSS ở A4 (Chương 2) là bản mở rộng

của bài này: cùng ý tưởng lan truyền ngược tự viết, thêm kiểm tra gradient bằng sai phân hữu hạn và so sánh với Keras, PyTorch. Kết quả ở đó cho thấy đúng vấn đề mà câu hỏi đổi ngưỡng nhắm tới: không có trọng số lớp, accuracy đạt 0,8372 nhưng recall lớp dương chỉ khoảng 0,17.

Chương 2

Mạng nơ-ron tích chập

Chương này đi từ mạng nơ-ron nhiều lớp (MLP) và thuật toán lan truyền ngược tới mạng tích chập (CNN), rồi tóm tắt các demo của Assignment 4 và 5.

2.1. Mạng nơ-ron nhiều lớp

2.1.1 Mạng là hợp của các hàm sơ cấp

Một lớp kết nối đầy đủ tính $h = \phi(Wx+b)$, với ϕ là hàm kích hoạt phi tuyến: ReLU $\max(0, z)$, sigmoid, tanh. Mạng L lớp là hợp của các hàm:

$$f(x) = f_L \circ f_{L-1} \circ \ldots \circ f_1(x), \qquad f_l(h) = \phi_l(W_l h + b_l).$$

Không có ϕ, hợp của các hàm tuyến tính vẫn là tuyến tính; hàm kích hoạt cho mạng khả năng xấp xỉ quan hệ phi tuyến. Lớp cuối của bài phân loại dùng softmax, và hàm mất mát là cross-entropy như hồi quy logistic (Chương 1).

2.1.2 Lan truyền ngược

Gradient của hàm mất mát theo tham số mỗi lớp được tính bằng quy tắc dây chuyền, đi ngược từ đầu ra. Với $a_l = W_l h_{l-1} + b_l$, $h_l = \phi(a_l)$ và $\delta_l = \partial \ell / \partial a_l$:

$$\delta_L = p - y \;\; \text{(softmax + cross-entropy)}, \qquad \delta_l = (W_{l+1}^\top \delta_{l+1}) \odot \phi'(a_l), \qquad \frac{\partial \ell}{\partial W_l} = \delta_l \, h_{l-1}^\top.$$

Khi tự cài đặt, gradient được kiểm bằng sai phân hữu hạn $\frac{\partial \ell}{\partial \theta} \approx \frac{\ell(\theta+\varepsilon) - \ell(\theta-\varepsilon)}{2\varepsilon}$; sai số tương đối cỡ 10^{-7} cho thấy cài đặt đúng.

2.1.3 Tối ưu và điều chuẩn

SGD cập nhật $\theta \leftarrow \theta - \eta\, g$ trên từng lô nhỏ. Adam giữ trung bình trượt của gradient và của bình phương gradient, nên mỗi tham số có tốc độ học riêng:

$$m \leftarrow \beta_1 m + (1-\beta_1) g, \quad v \leftarrow \beta_2 v + (1-\beta_2) g^2, \quad \theta \leftarrow \theta - \eta \, \frac{\hat{m}}{\sqrt{\hat{v}} + \epsilon},$$

với $\hat{m}, \hat{v}$ là bản đã hiệu chỉnh độ chệch. Các kỹ thuật điều chuẩn: dropout (tắt ngẫu nhiên một tỷ lệ đơn vị khi huấn luyện), weight decay, dừng sớm theo tập xác thực, và chuẩn hóa theo lô (batch normalization) đưa đầu vào mỗi lớp về trung bình 0, phương sai 1 trên lô.

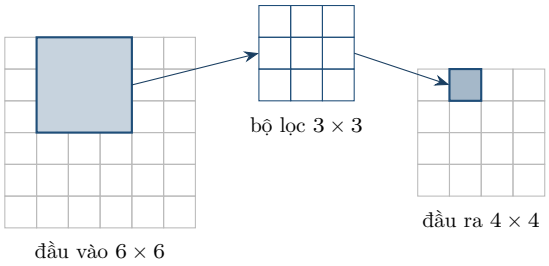
2.2. Phép tích chập

2.2.1 Định nghĩa

MLP nối mọi điểm ảnh với mọi đơn vị: một ảnh $32 \times 32 \times 3$ vào lớp 512 đơn vị cần hơn 1,5 triệu trọng số, và mô hình phải học lại cùng một mẫu (ví dụ một cạnh) ở mọi vị trí. Tích chập dùng một bộ lọc nhỏ trượt trên ảnh:

$$Y[i,j] = \sum_{c} \sum_{u=0}^{k-1} \sum_{v=0}^{k-1} K[c,u,v] \; X[c,\; is+u,\; js+v] + b.$$

Hai tính chất làm CNN hợp với ảnh: *kết nối cục bộ* (mỗi đầu ra chỉ nhìn một vùng $k \times k$) và *chia sẻ trọng số* (cùng bộ lọc ở mọi vị trí, nên một mẫu được nhận ra ở bất kỳ đâu).



Hình 2.1: Một bước tích chập: vùng 3×3 của đầu vào nhân từng phần tử với bộ lọc rồi cộng lại thành một ô của đầu ra (padding 0, stride 1).

2.2.2 Kích thước, pooling, vùng tiếp nhận

Với đầu vào cạnh n, bộ lọc k, padding p, bước nhảy s, cạnh đầu ra là

$$n_{\text{out}} = \left\lfloor \frac{n + 2p - k}{s} \right\rfloor + 1.$$

Số tham số của một lớp tích chập là $(k^2 C_{\text{in}} + 1)\, C_{\text{out}}$, không phụ thuộc kích thước ảnh. Pooling (cực đại hoặc trung bình trên cửa sổ 2×2, bước 2) giảm một nửa mỗi chiều không gian, giúp mô hình ít nhạy với dịch chuyển nhỏ. Xếp chồng các lớp làm *vùng tiếp nhận* tăng dần: lớp sâu nhìn được vùng lớn của ảnh gốc, nên học được các mẫu trừu tượng hơn (cạnh $\rightarrow$ hoa văn $\rightarrow$ bộ phận $\rightarrow$ vật thể).

2.2.3 Cài đặt bằng im2col

Vòng lặp trượt bộ lọc rất chậm trong Python. im2col trải mỗi vùng $k \times k \times C$ thành một cột của ma trận, nên toàn bộ phép tích chập trở thành một phép nhân ma trận và tận dụng được thư viện đại số tuyến tính. Đổi lại, bộ nhớ tăng khoảng k^2 lần.

Lớp `Conv2D` trong bản NumPy của A4 làm đúng như vậy. Ở chiều tiến, bộ lọc được trải thành ma trận `w2` kích thước $C_{\text{out}} \times (C_{\text{in}} k^2)$ rồi nhân với các cột của `im2col`. Ở chiều lùi, cùng ma trận cột cho gradient của trọng số; gradient theo đầu vào được tính trên dạng cột rồi `col2im` cộng dồn về đúng vị trí điểm ảnh, vì một điểm ảnh thuộc nhiều cửa sổ.

Đoạn mã 2.1. Tích chập bằng im2col, chiều tiến và chiều lùi. `ass_04/A4_02_TranXuanThanh_280/src/scratch_nn.py`, dòng 103–124.

```
    def forward(self, x):
        cols, oh, ow = im2col(x, self.k, self.k, self.stride, self.pad)
        w = self.params["W"]
        f = w.shape[0]
        w2 = w.reshape(f, -1)
        out = np.einsum("fc,ncl->nfl", w2, cols, optimize=True)
        out = out + self.params["b"].reshape(1, f, 1)
        self._cache = (x.shape, cols, oh, ow)
        return out.reshape(x.shape[0], f, oh, ow)

    def backward(self, dout):
        x_shape, cols, oh, ow = self._cache
        n, f = dout.shape[0], dout.shape[1]
        dout2 = dout.reshape(n, f, oh * ow)

        self.grads["b"] = dout2.sum(axis=(0, 2))
        dw2 = np.einsum("nfl,ncl->fc", dout2, cols, optimize=True)
        self.grads["W"] = dw2.reshape(self.params["W"].shape)

        w2 = self.params["W"].reshape(f, -1)
        dcols = np.einsum("fc,nfl->ncl", w2, dout2, optimize=True)
        return col2im(dcols, x_shape, self.k, self.k, self.stride, self.pad)
```

2.2.4 Một kiến trúc CNN điển hình

Các khối *tích chập – ReLU – (chuẩn hóa) – pooling* lặp lại, số kênh tăng dần khi kích thước không gian giảm; cuối cùng gộp toàn cục hoặc làm phẳng rồi qua các lớp kết nối đầy đủ và softmax. Grad-CAM giải thích dự đoán bằng cách lấy gradient của điểm lớp theo bản đồ đặc trưng của lớp tích chập cuối, cho thấy vùng ảnh mô hình dựa vào.

2.3. Demo các assignment

2.3.1 Assignment 4: một kiến trúc, ba bản cài đặt

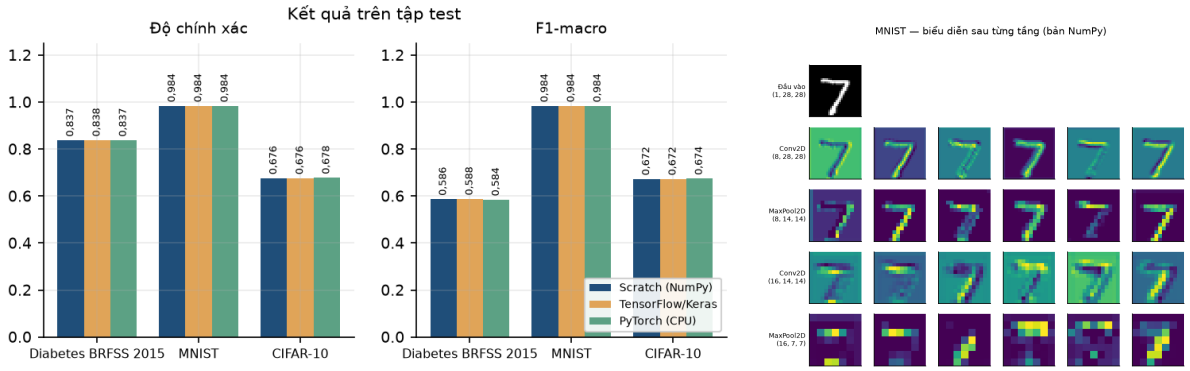
Bài toán. Cùng một mạng được cài bằng NumPy viết tay (tích chập bằng im2col, lan truyền ngược tự viết), Keras và PyTorch; ba bản dùng chung trọng số khởi tạo và thứ tự mẫu. Dữ liệu: BRFSS (MLP, 3.554 tham số), MNIST (CNN, 9.098 tham số), CIFAR-10 (CNN, 25.578 tham số). Kiểm tra gradient của bản NumPy đạt, sai số tương đối lớn nhất $2{,}3 \times 10^{-6}$.

Bảng 2.1: A4: accuracy trên tập kiểm tra (thời gian huấn luyện, giây)

Dữ liệu	NumPy	Keras	PyTorch (CPU)
Diabetes	0,8372 (2,0)	0,8376 (5,0)	0,8371 (3,2)
MNIST	0,9839 (212,4)	0,9840 (29,6)	0,9837 (48,8)
CIFAR-10	0,6762 (565,9)	0,6762 (55,2)	0,6777 (112,2)

Điểm chính.

- Cùng dữ liệu, trọng số khởi tạo và thứ tự mẫu thì ba bản khớp nhau tới chữ số thứ ba; dự đoán của bản NumPy và Keras trùng 99,94% trên MNIST và 96,22% trên CIFAR-10. Phần lệch còn lại đến từ phép cộng dấu phẩy động và cách Keras viết Adam.
- Bản NumPy nhanh nhất với MLP nhỏ nhưng chậm gấp khoảng 10 lần trên CIFAR-10: im2col tốn bộ nhớ và không tận dụng được nhân tối ưu của thư viện.
- Recall lớp dương của bài Diabetes chỉ khoảng 0,17 ở cả ba bản: vấn đề nằm ở hàm mục tiêu không có trọng số lớp, không ở thư viện.
- Khi chép trọng số giữa thư viện, thứ tự làm phẳng NHWC (Keras) và NCHW (PyTorch) phải được sắp lại; bỏ qua bước này làm logit lệch 6,2 trên MNIST.



Hình 2.2: A4: chỉ số của ba bản trên ba bộ dữ liệu (trái) và bản đồ đặc trưng của lớp tích chập trên MNIST (phải).

2.3.2 Assignment 5: Keras và PyTorch, 3 tầng và 5 tầng

Bài toán. 12 thí nghiệm: 3 bộ dữ liệu × 2 thư viện × 2 độ sâu, từ cùng một đặc tả kiến trúc. Dữ liệu: VinBigData X-quang ngực (15.000 ảnh 128×128, 29,3% bất thường), CIFAR-10 (60.000 ảnh), BRFSS (253.680 mẫu, dùng tích chập 1 chiều).

Bảng 2.2: A5: F1 trên tập kiểm tra, 3 tầng → 5 tầng

Dữ liệu	Keras	PyTorch
VinBigData X-quang	0,8049 → 0,8365	0,8155 → 0,8760
CIFAR-10	0,7276 → 0,6180	0,6851 → 0,7599
Diabetes BRFSS	0,4924 → 0,4906	0,4914 → 0,4856

Điểm chính.

- **Sâu hơn không tự động tốt hơn.** Năm tầng giúp ảnh X-quang (cấu trúc không gian thật) nhưng không giúp dữ liệu bảng; ở đó thêm tầng chủ yếu là thêm chi phí (số tham số tăng khoảng 10 lần).
- Con số giảm của Keras 5 tầng trên CIFAR-10 do mô hình dừng sớm sau 8 epoch: mạng sâu nhạy với tiêu chí dừng.
- So sánh thư viện chỉ có nghĩa khi ép kiến trúc giống nhau; trung bình Keras tốn khoảng 1,46 lần thời gian mỗi epoch so với PyTorch.
- Grad-CAM cho thấy hai mặt: ở một ca bất thường, vùng nóng phủ đúng thùy phổi có đám mờ; nhưng ở các ca khác vùng nóng nằm ngoài trường phổi (viền ảnh, phần mềm ngoài lồng ngực). Mô hình đang dùng một phần tín hiệu không liên quan đến bệnh, điều mà bảng chỉ số không cho thấy.

Một đặc tả, hai thư viện. Mỗi thư viện có một hàm dịch cùng danh sách khối của `arch.py` thành mô hình. Đoạn mã dưới đặt cạnh nhau phần dựng khối tích chập: thứ tự Conv → BatchNorm → ReLU → pooling, khởi tạo He và bias 0 giống nhau. Chỗ phải chú ý là tham số `momentum` của BatchNorm: Keras dùng 0,99 còn PyTorch dùng 0,01 cho cùng một hành vi, vì hai thư viện định nghĩa ngược nhau.

Đoạn mã 2.2. Khối tích chập trong bản Keras (trên) và bản PyTorch (dưới). `ass_05/A5_02_TranXuanThanh_280/src/keras_backend.py`, dòng 27–34; `torch_backend.py`, dòng 25–33.

```
    for b in blocks:
        if b.kind == "conv":
            x = Conv(b.channels, b.kernel, padding="same", use_bias=True,
                     kernel_initializer="he_normal", bias_initializer="zeros")(x)
            x = layers.BatchNormalization(epsilon=1e-5, momentum=0.99)(x)
            x = layers.ReLU()(x)
            if b.pool:
                x = Pool(2)(x)
```

```
        for b in blocks:
            if b.kind == "conv":
                conv = Conv(in_ch, b.channels, b.kernel, padding=b.kernel // 2, bias=True)
                nn.init.kaiming_normal_(conv.weight, nonlinearity="relu")
                nn.init.zeros_(conv.bias)
                layers += [conv, Norm(b.channels, eps=1e-5, momentum=0.01), nn.ReLU()]
                if b.pool:
                    layers.append(Pool(2))
                in_ch = b.channels
```

Grad-CAM. Hàm `cam_for` gắn một hook vào lớp `Conv2d` cuối để giữ bản đồ đặc trưng A và gradient của điểm lớp "bất thường" theo A. Trọng số mỗi kênh là trung bình gradient trên hai chiều không gian; bản đồ nhiệt là ReLU của tổng có trọng số các kênh, chuẩn hóa về $[0, 1]$.

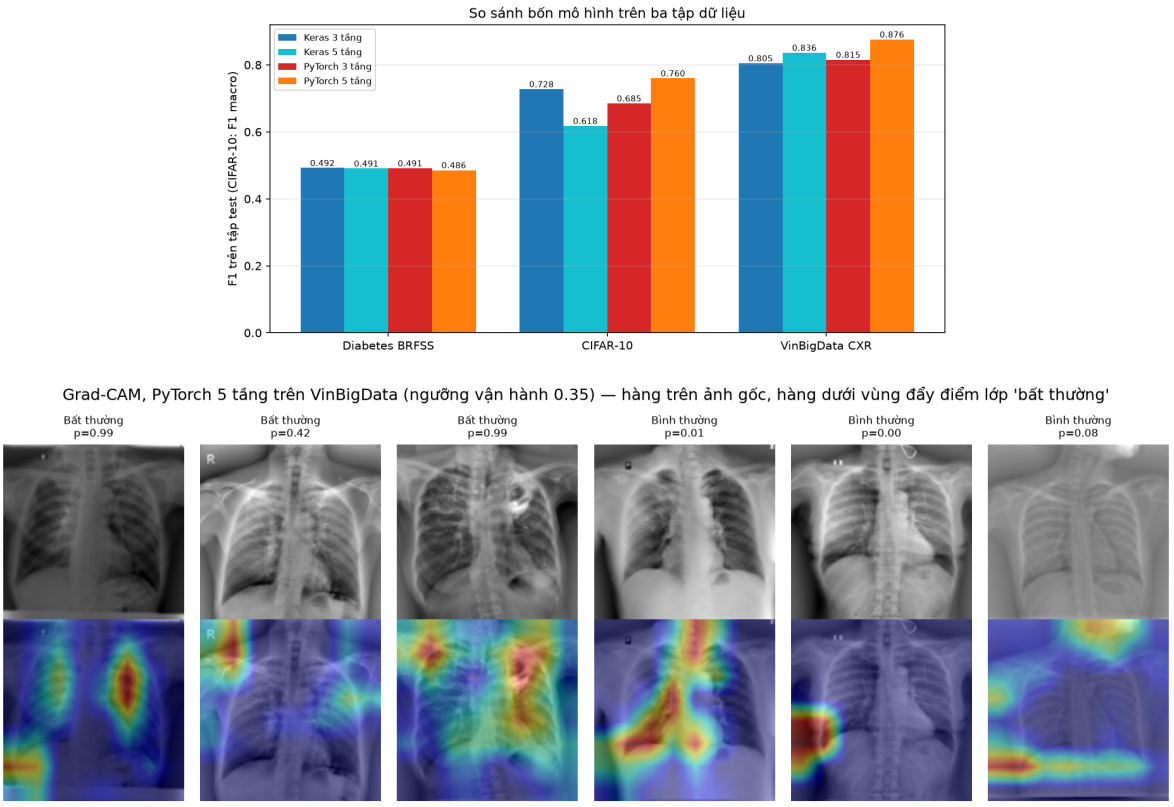
Đoạn mã 2.3. Tính Grad-CAM cho mô hình PyTorch. `ass_05/A5_02_TranXuanThanh_280/src/gradcam.py`, dòng 41–62.

```
    layer = _last_conv(model)
    activations: dict[str, torch.Tensor] = {}

    def fwd_hook(_m, _i, out):
        activations["a"] = out
        out.retain_grad()

    handle = layer.register_forward_hook(fwd_hook)
    model.eval()
    inp = torch.tensor(x[None], dtype=torch.float32)
    score = model(inp)[0, target]
    model.zero_grad()
    score.backward()
    handle.remove()

    a = activations["a"].detach()[0]            # (C, h, w)
    g = activations["a"].grad.detach()[0]
    weights = g.mean(dim=(1, 2), keepdim=True)
    cam = torch.relu((weights * a).sum(0)).numpy()
    if cam.max() > 0:
        cam = cam / cam.max()
    return cam
```



Hình 2.3: A5: F1 theo dữ liệu, thư viện và độ sâu (trên); Grad-CAM của mô hình PyTorch 5 tầng, ba cột trái là ca bất thường, ba cột phải là ca bình thường (dưới).

2.3.3 CNN trên dữ liệu bảng trong project

Project (mục 4) thử lại câu hỏi của A5 với đủ ba seed: CNN-1D đọc 19 đặc trưng BRFSS như một chuỗi token, đạt macro-F1 kiểm tra 0,4309 ± 0,0018 so với 0,4260 ± 0,0024 của MLP ở

bài ba nhóm, nhưng huấn luyện lâu hơn khoảng 56 lần. Kết luận giống A5: với dữ liệu bảng, lợi ích của tích chập rất nhỏ.

Chương 3

Mạng nơ-ron hồi quy

Ảnh có cấu trúc không gian; văn bản, giá cổ phiếu hay chuỗi đường huyết có cấu trúc thời gian. Chương này trình bày mạng nơ-ron hồi quy (RNN), cách huấn luyện và giới hạn của nó, rồi tóm tắt demo của Assignment 6 và phần RNN của Assignment 7.

3.1. Dữ liệu chuỗi

Một chuỗi $x_1, x_2, \dots, x_T$ có thứ tự, các phần tử phụ thuộc nhau, và độ dài có thể khác nhau giữa các mẫu. Các dạng bài:

- **nhiều–một**: đọc cả chuỗi, ra một nhãn (phân loại cảm xúc bình luận);
- **nhiều–nhiều**: mỗi bước một đầu ra (gán nhãn từ loại);
- **dự báo**: từ w bước gần nhất dự đoán bước tiếp theo (giá ngày mai, số đơn hàng giờ tới).

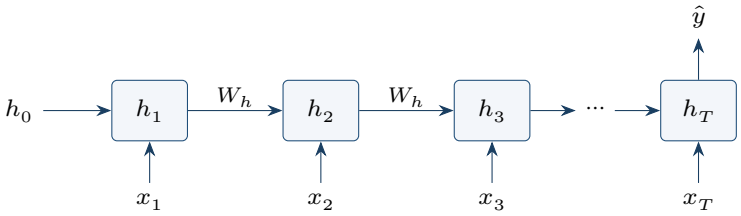
Với chuỗi thời gian, dữ liệu được cắt bằng cửa sổ trượt độ dài w; tập huấn luyện, xác thực, kiểm tra phải chia theo thời gian (quá khứ $\rightarrow$ tương lai), và thống kê chuẩn hóa chỉ tính trên đoạn huấn luyện. Với văn bản, mỗi từ (hoặc âm tiết) được ánh xạ thành một vector học được qua lớp embedding.

3.2. Cấu trúc RNN

RNN đọc chuỗi từng bước và mang theo một trạng thái ẩn tóm tắt những gì đã đọc:

$$h_t = \tanh(W_x x_t + W_h h_{t-1} + b), \qquad \hat{y} = \mathrm{softmax}(W_y h_T + c).$$

Cùng bộ tham số (W_x, W_h, b) được dùng ở mọi bước, nên số tham số không phụ thuộc độ dài chuỗi. Dàn theo thời gian, RNN là một mạng sâu T lớp có trọng số chia sẻ; mỗi bước là hợp của các hàm sơ cấp: nhân ma trận, cộng, tanh.



Hình 3.1: RNN dàn theo thời gian, dạng nhiều–một. Mọi ô dùng chung W_x, W_h.

3.3. Huấn luyện

3.3.1 Lan truyền ngược theo thời gian

BPTT (backpropagation through time) áp dụng lan truyền ngược trên mạng đã dàn; gradient của W_h là tổng đóng góp ở mọi bước. Gradient từ bước T về bước t đi qua tích các Jacobian:

$$\frac{\partial h_T}{\partial h_t} = \prod_{k=t+1}^{T} \mathrm{diag}(1 - h_k^2)\, W_h.$$

3.3.2 Gradient tắt dần và bùng nổ

Mỗi thừa số có đạo hàm tanh không quá 1. Nếu chuẩn của W_h nhỏ hơn 1, tích giảm theo cấp số nhân với khoảng cách $T - t$: tín hiệu từ đầu chuỗi gần như không tới được tham số, nên RNN khó học phụ thuộc xa (*gradient tắt dần*). Nếu lớn hơn 1, tích tăng theo cấp số nhân (*gradient bùng nổ*), làm huấn luyện mất ổn định. Bùng nổ được xử lý bằng cắt gradient (gradient clipping): nếu $\|g\| > \tau$ thì $g \leftarrow \tau\, g/\|g\|$. Tắt dần không sửa được bằng cách cắt, mà cần đổi kiến trúc (Chương 4).

3.3.3 Cài đặt

Cài đặt tay bằng NumPy gồm vòng lặp tiến qua T bước, lưu các h_t, rồi vòng lặp lùi cộng dồn gradient. Các thư viện cung cấp sẵn: Keras `SimpleRNN`, PyTorch `nn.RNN`. Với lô gồm các chuỗi dài khác nhau, chuỗi được đệm (padding) tới cùng độ dài và dùng mặt nạ để bỏ các bước đệm.

Bản NumPy của A6 cài đúng hai vòng lặp này. `_forward` lưu mọi trạng thái `states`; trong `fit`, vòng `for t` chạy ngược từ bước cuối, cộng dồn gradient của W_x, W_h, b ở mọi bước, rồi nhân `dh` với W_h và $1 - h_t^2$ để đẩy gradient về bước trước, đúng thừa số trong công thức BPTT ở trên. Dòng cuối cắt từng phần tử gradient về $[-1, 1]$ trước khi cập nhật.

Đoạn mã 3.1. RNN viết tay: lan truyền tiến và BPTT. `ass_06/A6_02_TranXuanThanh_280/src/rnn_scratch.py`, dòng 17–22, 36–51.

```
def _forward(self, sequence):
    states = [np.zeros(self.h, np.float32)]
    for xt in sequence:
        states.append(np.tanh(xt @ self.Wxh + states[-1] @ self.Whh + self.bh))
    pred = states[-1] @ self.Why + self.by
    return float(pred[0]), states
# ...
            for idx in ids:
                pred, states = self._forward(X[idx])
                err = pred - float(y[idx])
                epoch_loss += err * err
                grads[3] += states[-1][:, None] * err
                grads[4] += err
                dh = (self.Why[:, 0] * err) * (1 - states[-1] ** 2)
                for t in range(len(X[idx])-1, -1, -1):
                    grads[0] += np.outer(X[idx, t], dh)
                    grads[1] += np.outer(states[t], dh)
                    grads[2] += dh
                    dh = (self.Whh @ dh) * (1 - states[t] ** 2)
            scale = 1 / max(1, len(ids))
            params = (self.Wxh, self.Whh, self.bh, self.Why, self.by)
            for p, g in zip(params, grads):
                p -= self.lr * np.clip(g * scale, -1, 1)
```

3.4. Demo các assignment

3.4.1 Assignment 6: dự báo số giao dịch theo giờ

Bài toán. Từ 24 giờ gần nhất (số lượt xem, thêm vào giỏ, giao dịch) của Retailrocket, dự báo số giao dịch giờ kế tiếp. Dữ liệu gồm 3.312 giờ, chia theo thời gian 2.318 / 497 / 497; mục tiêu biến đổi bằng $\log(1+x)$. Ba bản RNN 16 đơn vị: NumPy viết tay (SGD), Keras `SimpleRNN`, PyTorch `nn.RNN`.

Bảng 3.1: A6: sai số trên tập kiểm tra (đơn hàng/giờ)

Mô hình	MAE	RMSE
Mốc: trung bình tập huấn luyện	4,44	5,89
RNN NumPy	2,96	4,51
Keras SimpleRNN	2,56	3,80
PyTorch nn.RNN	2,52	3,77

Điểm chính.

- Cả ba RNN thắng mốc; PyTorch giảm MAE khoảng 43%. Bản NumPy dùng SGD còn hai thư viện dùng Adam, nên đây là so sánh cách cài đặt, không phải phép đo có kiểm soát về thư viện.
- Keras và PyTorch chênh nhau ít hơn mức bất định; một seed và một đoạn kiểm tra không đủ để xếp hạng hai thư viện.
- Giới hạn: chỉ 138 ngày dữ liệu, không có đặc trưng lịch hay giá; báo cáo đề xuất thử GRU/LSTM cho phụ thuộc dài hơn.

3.4.2 Assignment 7, phần RNN: RNN là hợp của các hàm sơ cấp

A7 viết mỗi bước RNN thành hợp của 5 hàm sơ cấp, kiểm chứng với PyTorch và Keras, và chạy trên hai bài: phân loại cảm xúc 3 lớp cho bình luận thegioididong (9.030 bình luận, chia 6.321 / 1.354 / 1.355) và dự báo giá cổ phiếu FPT (4.071 phiên, cửa sổ 30). RNN đạt macro-F1 0,592 ± 0,014 (PyTorch) và 0,594 ± 0,023 (Keras) trên bài cảm xúc, qua 5 seed.

Trong mã của A7, mỗi hàm sơ cấp là một hàm Python riêng (`lin`, `add`, `shift`, `tanh`), và một bước RNN là lời gọi lồng nhau của chúng. Jacobian $\partial h_t/\partial h_{t-1}$ được ghép từ Jacobian của từng hàm: `lin` cho U, `add` và `shift` cho ma trận đơn vị, `tanh` cho $\mathrm{diag}(1-h_t^2)$. Đây là thừa số được nhân liên tiếp trong công thức gradient tắt dần.

Đoạn mã 3.2. Bước RNN là hợp của 5 hàm sơ cấp. `ass_07/A7_02_TranXuanThanh_280/src/elementary.py`, dòng 130–151.

```
def rnn_step(p: CellParams, x, h, tape=_NULL):
    """f_t(h) = tanh(shift_b(add(lin_W(x_t), lin_U(h))))  - 5 hàm sơ cấp."""
    return tanh(shift(add(lin(p.W, x, tape), lin(p.U, h, tape), tape), p.b, tape), tape)


def rnn_forward(p: CellParams, xs: np.ndarray, h0=None, tape=_NULL):
    h = np.zeros(p.H) if h0 is None else h0
    hs = []
    for x in xs:
        h = rnn_step(p, x, h, tape)
        hs.append(h)
    return np.stack(hs)


def rnn_step_jacobian(p: CellParams, h_new):
    return (1.0 - h_new ** 2)[:, None] * p.U
```

Thí nghiệm đo độ lớn gradient theo khoảng cách thời gian xác nhận hiện tượng gradient tắt dần đã phân tích ở trên: với RNN PyTorch, gradient về tới bước đầu chuỗi chỉ còn khoảng 10^{-3} sau 39 bước. Kết quả này là điểm xuất phát của Chương 4.

Chương 4

Long Short-Term Memory

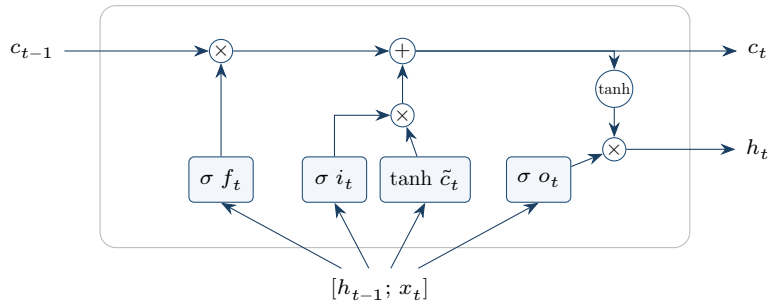
LSTM sửa giới hạn chính của RNN: gradient tắt dần trên chuỗi dài. Chương này trình bày cấu trúc ô LSTM, các biến thể, rồi tóm tắt phần so sánh RNN–LSTM của Assignment 7 và phần LSTM trong project.

4.1. Ô nhớ và các cổng

LSTM thêm một *ô nhớ* c_t bên cạnh trạng thái ẩn h_t, và ba cổng điều khiển việc ghi, xóa, đọc ô nhớ. Với $z_t = [h_{t-1}; x_t]$:

$$
\begin{aligned}
f_t &= \sigma(W_f z_t + b_f) && \text{cổng quên: giữ bao nhiêu của } c_{t-1} \\
i_t &= \sigma(W_i z_t + b_i) && \text{cổng vào: ghi bao nhiêu thông tin mới} \\
\tilde{c}_t &= \tanh(W_c z_t + b_c) && \text{thông tin ứng viên} \\
c_t &= f_t \odot c_{t-1} + i_t \odot \tilde{c}_t && \text{cập nhật ô nhớ} \\
o_t &= \sigma(W_o z_t + b_o) && \text{cổng ra} \\
h_t &= o_t \odot \tanh(c_t). &&
\end{aligned}
$$

Các cổng dùng sigmoid nên có giá trị trong $(0, 1)$, hoạt động như van mở một phần. Mỗi cổng vẫn là hợp của các hàm sơ cấp: nhân ma trận, cộng, sigmoid, tanh, nhân từng phần tử.



Hình 4.1: Ô LSTM. Đường trên cùng là ô nhớ: chỉ đi qua phép nhân với cổng quên và phép cộng.

Vì sao LSTM giữ được phụ thuộc xa. Theo đường ô nhớ, $\partial c_t / \partial c_{t-1} = f_t$ (bỏ qua đường gián tiếp qua các cổng). Khi cổng quên gần 1, gradient đi qua nhiều bước gần như không suy giảm, khác với tích các Jacobian chứa W_h và $\tanh'$ của RNN. Phép *cộng* trong cập nhật ô nhớ là điểm mấu chốt; khởi tạo b_f dương giúp cổng quên mở từ đầu.

Bản cài đặt của A7 viết một bước LSTM bằng các hàm sơ cấp đã dùng cho RNN ở Chương 3, thêm `sigmoid`, `mul` và `split`. Một phép nhân ma trận tính tiền kích hoạt của cả bốn khối, rồi `split` tách thành i, f, g, o theo thứ tự trùng với PyTorch và Keras (g là $\tilde{c}_t$). Hai dòng cuối là cập nhật ô nhớ và trạng thái ẩn.

Đoạn mã 4.1. Một bước LSTM, 17 lần gọi hàm sơ cấp. `ass_07/A7_02_TranXuanThanh_280/src/elementary.py`, dòng 155–171.

```
def lstm_step(p: CellParams, x, h, c, tape=_NULL):
    H = p.H
    z = shift(add(lin(p.W, x, tape), lin(p.U, h, tape), tape), p.b, tape)
    i = sigmoid(split(z, 0, H, tape), tape)
    f = sigmoid(split(z, 1, H, tape), tape)
    g = tanh(split(z, 2, H, tape), tape)
```

```
o = sigmoid(split(z, 3, H, tape), tape)
c_new = add(mul(f, c, tape), mul(i, g, tape), tape)
h_new = mul(o, tanh(c_new, tape), tape)
return h_new, c_new, (i, f, g, o)
```

Jacobian của bước LSTM được ghép theo cùng cách. Khối `dc_dc` $= \mathrm{diag}(f)$ là đường ô nhớ: không có thừa số U nào, nên gradient đi qua nhiều bước chỉ bị nhân với các giá trị cổng quên.

Đoạn mã 4.2. Jacobian của một bước LSTM. `ass_07/A7_02_TranXuanThanh_280/src/elementary.py`, dòng 197–210.

```
H = p.H
i, f, g, o = gates
Ui, Uf, Ug, Uo = (p.U[k * H:(k + 1) * H] for k in range(4))
di = (i * (1 - i))[:, None] * Ui
df = (f * (1 - f))[:, None] * Uf
dg = (1 - g ** 2)[:, None] * Ug
do = (o * (1 - o))[:, None] * Uo
dc_dh = c_prev[:, None] * df + g[:, None] * di + i[:, None] * dg
dc_dc = np.diag(f)
tc = np.tanh(c_new)
dh_dcn = (o * (1 - tc ** 2))[:, None]
dh_dh = tc[:, None] * do + dh_dcn * dc_dh
dh_dc = dh_dcn * dc_dc
return np.block([[dh_dh, dh_dc], [dc_dh, dc_dc]])
```

4.2. Biến thể và chi phí

GRU gộp cổng quên và cổng vào thành một cổng cập nhật u_t, bỏ ô nhớ riêng:

$$u_t = \sigma(W_u z_t + b_u), \qquad r_t = \sigma(W_r z_t + b_r),$$
$$\tilde{h}_t = \tanh(W_h[\, r_t \odot h_{t-1};\, x_t\,] + b_h), \qquad h_t = (1 - u_t) \odot h_{t-1} + u_t \odot \tilde{h}_t.$$

Cổng đặt lại r_t quyết định dùng bao nhiêu trạng thái cũ khi tính ứng viên. GRU có ba khối trọng số thay vì bốn, nhanh hơn và thường cho kết quả gần LSTM.

LSTM hai chiều chạy một LSTM từ trái sang phải và một từ phải sang trái, rồi nối trạng thái của hai chiều; hợp với bài toán có toàn bộ chuỗi ngay từ đầu (phân loại văn bản), không dùng được cho dự báo thời gian thực.

Số tham số. Với đầu vào d chiều và trạng thái h chiều:

Ô	Số tham số
RNN	$h(h+d)+h$
GRU	$3\,[h(h+d)+h]$
LSTM	$4\,[h(h+d)+h]$

LSTM tốn gấp khoảng bốn lần RNN cùng kích thước, và mỗi bước vẫn phải chờ bước trước nên không song song hóa theo thời gian được. Giới hạn này dẫn tới Transformer, kiến trúc dùng cơ chế chú ý thay cho hồi quy và là nền của các mô hình ngôn ngữ lớn ở Chương 5 và 6.

4.3. Demo các assignment

4.3.1 Assignment 7: so sánh RNN và LSTM

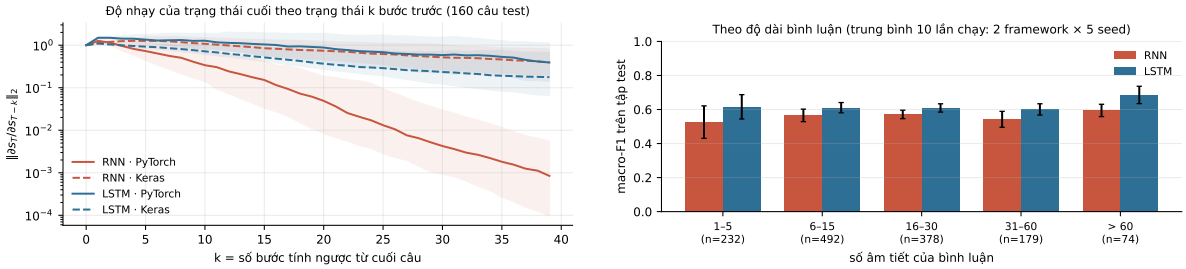
LSTM được viết thành hợp của 17 hàm sơ cấp mỗi bước, kiểm chứng với PyTorch và Keras, rồi so với RNN trên cùng hai bài của Chương 3; mỗi cấu hình chạy 5 seed (40 lần chạy).

Bảng 4.1: A7: RNN và LSTM, trung bình ± độ lệch chuẩn qua 5 seed

Ô	Thư viện	Macro-F1 cảm xúc	RMSE giá FPT (đồng)	Tham số hồi quy
Giá mai = giá nay		–	1.645	–
RNN	PyTorch	0,592 ± 0,014	1.650 ± 4	8.320
RNN	Keras	0,594 ± 0,023	1.656 ± 5	
LSTM	PyTorch	0,635 ± 0,009	1.648 ± 1	33.280
LSTM	Keras	0,651 ± 0,019	1.649 ± 1	

Điểm chính.

- **Văn bản: LSTM tốt hơn rõ.** Macro-F1 trung bình 0,643 so với 0,593 của RNN, cải thiện nhiều nhất ở lớp trung tính. Đổi lại LSTM có gấp 4 lần tham số hồi quy (đúng với công thức ở trên) và chậm hơn khoảng 2,8 lần.
- **Gradient**: sau 39 bước, gradient của RNN PyTorch còn khoảng 10^{-3}, của LSTM vẫn khoảng 0,2–0,4. Đây là bằng chứng thực nghiệm cho lập luận về đường ô nhớ.
- **Giá cổ phiếu: không mô hình nào thắng mốc ngây thơ** "giá mai = giá nay"; dự báo gần như là một hằng số, tỷ lệ đoán đúng hướng 46,6–50,1%. Kiến trúc tốt hơn không bù được việc dữ liệu không chứa tín hiệu dự báo.
- PyTorch và Keras cho kết quả gần như nhau khi cùng kiến trúc.



Hình 4.2: A7: độ lớn gradient theo khoảng cách thời gian (trái) và macro-F1 theo độ dài bình luận (phải).

4.3.2 LSTM trên dữ liệu bảng trong project

Project (mục 4) dùng LSTM hai chiều đọc 19 đặc trưng BRFSS như 19 token. Ở bài Type 1/2 (22.027 dòng), LSTM đạt macro-F1 kiểm tra 0,5745 ± 0,0157 so với 0,5422 ± 0,0061 của MLP và được chọn làm mô hình của agent dự đoán; ở bài ba nhóm, LSTM (0,4293) chỉ ngang CNN-1D và MLP. Độ lệch giữa các seed của bài Type 1/2 lớn, nên khoảng cách này cần thêm seed để khẳng định.

Để LSTM và CNN-1D đọc được một hàng dữ liệu bảng, lớp `FeatureTokens` biến đặc trưng thứ i thành token $x_i w_i + e_i$: giá trị đã chuẩn hóa nhân với một vector học được của riêng đặc trưng đó, cộng với embedding vị trí. `LSTMNet` chạy LSTM hai chiều trên 19 token và nối trạng thái cuối của hai chiều làm đầu vào cho lớp phân loại. `CNN1D` dùng chung lớp token, thay LSTM bằng hai lớp `Conv1d` kernel 3.

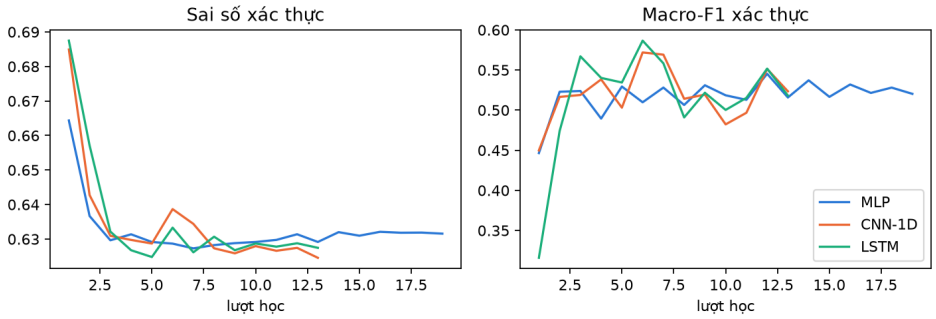
Đoạn mã 4.3. Token hóa đặc trưng và LSTM hai chiều của project. `tieuluan_final/project/backend/app/nets.py`, dòng 22–29, 47–56.

```
class FeatureTokens(nn.Module):
    def __init__(self, n_features: int, dim: int):
        super().__init__()
        self.scale = nn.Parameter(torch.randn(n_features, dim) * 0.1)
```

```
        self.position = nn.Parameter(torch.randn(n_features, dim) * 0.1)

    def forward(self, x: torch.Tensor) -> torch.Tensor:  # (B, F) -> (B, F, D)
        return x.unsqueeze(-1) * self.scale + self.position
# ...
class LSTMNet(nn.Module):
    def __init__(self, n_in: int, n_out: int, dim: int = 16, hidden: int = 32, dropout: float = 0.15):
        super().__init__()
        self.tokens = FeatureTokens(n_in, dim)
        self.lstm = nn.LSTM(dim, hidden, batch_first=True, bidirectional=True)
        self.head = nn.Sequential(nn.Dropout(dropout), nn.Linear(2 * hidden, 32), nn.ReLU(), nn.Linear(32, n_out))

    def forward(self, x: torch.Tensor) -> torch.Tensor:
        _, (h, _) = self.lstm(self.tokens(x))  # h: (2, B, H), last state of each direction
        return self.head(torch.cat([h[0], h[1]], dim=1))
```



Hình 4.3: Project: sai số và macro-F1 tập xác thực theo lượt học, bài Type 1/2.

Chương 5

Knowledge Base và RAG

Các chương trước xây mô hình học từ dữ liệu có nhãn: mô hình nhận một vector đặc trưng và trả về một nhãn hoặc một con số. Chương này giải quyết một loại bài toán khác: trả lời câu hỏi bằng ngôn ngữ tự nhiên dựa trên một kho tài liệu, sao cho mỗi ý trong câu trả lời chỉ được về một đoạn văn cụ thể. Hai thành phần chính là kho tri thức (Knowledge Base, KB) và kỹ thuật sinh câu trả lời có truy xuất (Retrieval-Augmented Generation, RAG).

Toàn bộ hệ thống mô tả trong chương được cài đặt và đo trên bài toán tra cứu thông tin về đái tháo đường (mã nguồn ở thư mục `ass_08/A8_02_TranXuanThanh_280`). Chương này cũng là mục 5 của project hệ đa agent: ở Chương 6, toàn bộ pipeline RAG trở thành một công cụ của agent tra cứu tri thức.

5.1. Vì sao cần KB và RAG

5.1.1 Giới hạn của mô hình ngôn ngữ lớn

Mô hình ngôn ngữ lớn (LLM) học phân bố của văn bản từ một kho dữ liệu rất lớn, rồi sinh văn bản bằng cách dự đoán token tiếp theo. Cách học này cho mô hình khả năng viết trôi chảy, nhưng có bốn giới hạn khi dùng để trả lời câu hỏi y tế:

- **Bịa thông tin (hallucination).** Mô hình có thể sinh ra câu đúng ngữ pháp, nghe hợp lý nhưng sai sự thật, ví dụ một ngưỡng xét nghiệm hay một liều thuốc không có trong tài liệu nào [2].
- **Tri thức đóng băng.** Mô hình chỉ biết những gì có trong dữ liệu huấn luyện; hướng dẫn điều trị ban hành sau thời điểm đó không có trong mô hình.
- **Không chỉ được nguồn.** Tri thức nằm phân tán trong hàng tỷ tham số, nên mô hình không thể cho biết một câu trả lời đến từ tài liệu nào.
- **Không biết tài liệu riêng.** Hướng dẫn của Bộ Y tế Việt Nam hay tài liệu nội bộ của một bệnh viện có thể không có, hoặc có rất ít, trong dữ liệu huấn luyện.

5.1.2 Hai cách bổ sung tri thức

Có hai cách đưa tri thức mới vào hệ thống: tinh chỉnh (fine-tuning) mô hình trên tài liệu mới, hoặc giữ nguyên mô hình và đưa tài liệu liên quan vào lời nhắc mỗi lần hỏi. Cách thứ hai là RAG [1].

Bảng 5.1: So sánh tinh chỉnh và RAG cho bài toán tra cứu y tế

Tiêu chí	Tinh chỉnh	RAG
Cập nhật tri thức	Huấn luyện lại	Thêm tài liệu vào kho, nạp lại
Chỉ ra nguồn	Không	Có, tới từng đoạn văn
Chi phí	GPU, dữ liệu huấn luyện dạng hỏi–đáp	Chỉ mục vector, một lần gọi mô hình mỗi câu
Kiểm soát bịa	Yếu	Kiểm được bằng code: ý nào không có nguồn thì bỏ
Khi nào phù hợp	Đổi văn phong, định dạng đầu ra	Tri thức thay đổi, cần trích dẫn

Yêu cầu đặt ra. Với thông tin y tế, một câu khuyến nghị bị tách khỏi điều kiện áp dụng của nó có thể gây hiểu sai. Hệ thống trong chương này vì vậy đặt hai yêu cầu: (1) *mọi ý trong câu trả lời phải chỉ được về một đoạn văn cụ thể trong một tài liệu cụ thể*; (2) khi kho không có thông tin, hệ thống phải nói rõ là không có thay vì đoán.

5.2. Knowledge Base

5.2.1 Khái niệm

Knowledge Base là kho tri thức được tổ chức để chương trình truy vấn được. Trong hệ thống này, KB lưu tri thức ở hai dạng bổ sung cho nhau:

- **Văn bản và vector.** Tài liệu được chia thành đoạn, mỗi đoạn được mã hóa thành một vector. Dạng này giữ nguyên câu chữ của nguồn, nên dùng để trích dẫn, và cho phép tìm đoạn gần nghĩa với câu hỏi.
- **Đồ thị tri thức.** Các thực thể (bệnh, triệu chứng, yếu tố nguy cơ, khuyến nghị, nhóm thuốc) là nút, và quan hệ giữa chúng là cạnh có hướng, ví dụ `(Đái tháo đường típ 2)-[HAS_RISK_FACTOR]->(Thừa cân)` [6]. Dạng này cho phép đi từ một dữ kiện tới các dữ kiện liên quan và tới mọi đoạn văn ủng hộ dữ kiện đó.

Hai dạng được nối với nhau: mỗi dữ kiện trên đồ thị có cạnh `SUPPORTED_BY` trỏ tới đoạn văn chứa câu trích làm bằng chứng. Cả hai dạng cùng nằm trong một cơ sở dữ liệu Neo4j, dưới tên `KB_diabet`.

5.2.2 Vì sao chọn Neo4j

Neo4j là hệ quản trị cơ sở dữ liệu đồ thị, truy vấn bằng ngôn ngữ Cypher. Từ phiên bản 5, Neo4j có chỉ mục vector [7], nên vector của đoạn văn được lưu ngay trên thuộc tính của nút `Chunk`. Một truy vấn có thể vừa tìm đoạn gần nghĩa bằng vector, vừa đi theo cạnh tới dữ kiện và tài liệu nguồn, mà không phải đồng bộ hai cơ sở dữ liệu riêng.

5.3. Xây KB từ link và file PDF

5.3.1 Nguồn tài liệu

Danh sách nguồn nằm trong `pipeline/sources.yaml`. Mỗi nguồn khai báo mã, địa chỉ, đơn vị xuất bản, ngôn ngữ và loại tệp (`html` hoặc `pdf`). Tiêu chí chọn nguồn: đơn vị xuất bản là cơ quan y tế, bệnh viện hoặc hiệp hội chuyên ngành; nội dung công khai; cho phép tải tự động.

Kho gồm 35 tài liệu của 16 đơn vị xuất bản. Nguồn tiếng Việt gồm ba hướng dẫn chẩn đoán và điều trị của Bộ Y tế (đái tháo đường típ 2 [8], tiền đái tháo đường, đái tháo đường típ 1 ở trẻ em), trang của các bệnh viện và Viện Dinh dưỡng Quốc gia. Nguồn tiếng Anh gồm NIDDK, WHO, NHS, NDSS, MedlinePlus, Diabetes UK, Hiệp hội Đái tháo đường Hoa Kỳ và Bộ Y tế Hồng Kông.

Bảng 5.2: Tài liệu và đoạn văn theo loại nguồn

Loại nguồn	Ngôn ngữ	Tài liệu	Đoạn văn
Link (trang web)	tiếng Việt	10	55
	tiếng Anh	18	91
File PDF	tiếng Việt	3	218
	tiếng Anh	4	60
Tổng		35	424

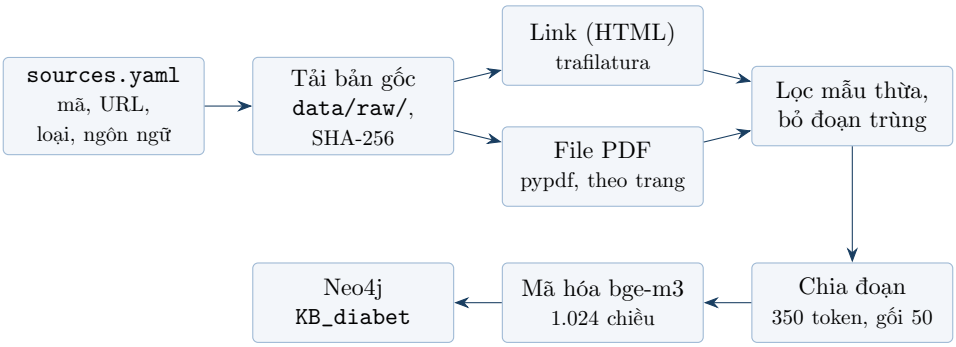
Link chiếm phần lớn số tài liệu (28 trên 35), nhưng PDF chiếm phần lớn nội dung (278 trên 424 đoạn). Lý do là PDF ở đây là các hướng dẫn chuyên môn dài hàng chục trang: riêng hướng dẫn đái tháo đường típ 2 của Bộ Y tế cho 121 đoạn, hướng dẫn típ 1 ở trẻ em cho 85 đoạn, còn một trang web thường chỉ cho 3 đến 8 đoạn.

Bảng 5.3: Số tài liệu theo đơn vị xuất bản

Đơn vị xuất bản	Tài liệu
NIDDK	5
NHS	4
American Diabetes Association	3
Bộ Y tế	3
Bệnh viện TWQĐ 108	2
Bệnh viện Tâm Anh	2
Diabetes UK	2
Hong Kong Department of Health	2
MedlinePlus	2
NDSS	2
Vinmec	2
WHO	2
Medlatec	1
Nhà thuốc Long Châu	1
Sức khỏe & Đời sống (Bộ Y tế)	1
Viện Dinh dưỡng Quốc gia	1

Nguồn không đưa vào được. Các trang của CDC (Hoa Kỳ) trả mã lỗi 403 cho chương trình tải tự động; hệ thống tôn trọng lệnh chặn này và không đưa CDC vào kho. Đường dẫn gốc của các quyết định Bộ Y tế trên cổng kcb.vn trả lỗi 404, nên văn bản được lấy từ bản đăng lại trên website bệnh viện, cùng số quyết định. Một trang giải thích HbA1c bị loại vì ghi nhầm ngưỡng 5,7–6,4% là đái tháo đường, trong khi đây là ngưỡng tiền đái tháo đường. Ví dụ này cho thấy bước chọn nguồn cần người đọc kiểm tra, không thể giao hết cho chương trình.

5.3.2 Hai luồng thu thập



Hình 5.1: Hai luồng thu thập link và PDF gặp nhau ở bước làm sạch.

Tải bản gốc. Chương trình `pipeline/fetch.py` tải từng nguồn vào `data/raw/` và ghi mã băm SHA-256 của tệp. Với nguồn khai báo là PDF, chương trình kiểm tra phần đầu tệp có chữ ký

%PDF; một số máy chủ trả trang HTML báo lỗi kèm mã 200, và bước kiểm này chặn trường hợp đó. Lần chạy sau bỏ qua tệp đã có cùng mã băm, nên chạy lại không tải lại và cho cùng kết quả.

Luồng link. Trang web chứa nhiều phần không phải nội dung: menu, quảng cáo, nút chia sẻ, chân trang. Thư viện trafilatura tách phần thân bài dựa trên cấu trúc thẻ HTML và mật độ chữ. Sau đó chương trình bỏ các dòng khớp danh sách mẫu trong `pipeline/boilerplate.txt`, ví dụ "đặt lịch khám", số tổng đài, lời mời tải ứng dụng.

Luồng PDF. PDF lưu chữ theo vị trí trên trang, không theo đoạn văn. Chương trình `pipeline/clean_chunk.py` đọc từng trang bằng pypdf rồi ghép dòng thành đoạn theo ba quy tắc: một dòng mở đoạn mới khi dòng trước kết thúc bằng dấu câu và dòng này bắt đầu bằng chữ hoa, hoặc khi dòng này là gạch đầu dòng; chữ bị ngắt bằng dấu gạch nối cuối dòng được nối lại; còn lại thì nối vào đoạn đang có. PDF tiếng Việt được trích ở chế độ giữ bố cục (`extraction_mode="layout"`) để giữ thứ tự dòng. Dòng xuất hiện ở ít nhất 3 trang hoặc 30% số trang (sau khi thay chữ số bằng ký hiệu chung) được coi là đầu trang, chân trang hay số trang, và bị bỏ.

Hai dòng đầu của hàm dưới đây tìm các dòng lặp lại (sau khi thay chữ số bằng `#`); biến `starts_new` là quy tắc mở đoạn mới, nhánh `elif` nối chữ bị ngắt bằng dấu gạch nối, nhánh `else` nối dòng vào đoạn đang có.

Đoạn mã 5.1. Ghép dòng PDF thành đoạn văn. `ass_08/A8_02_TranXuanThanh_280/pipeline/clean_chunk.py`, dòng 118–135.

```
def pdf_paragraphs(pages: list[list[str]]) -> list[str]:
    seen = Counter(normalize(re.sub(r"\d+", "#", ln)) for page in pages for ln in set(page))
    repeated = {k for k, n in seen.items() if n >= max(3, 0.3 * len(pages))}
    paragraphs: list[str] = []
    for page in pages:
        for raw in page:
            line = normalize(raw)
            if not line or normalize(re.sub(r"\d+", "#", line)) in repeated:
                continue
            line = re.sub(r"^[\uf0b7\uf0a7\uf0d8\uf02d]\s*", "- ", line)
            starts_new = not paragraphs or BULLET.match(line) or (paragraphs[-1].endswith(TERMINAL) and
            line[0].isupper())
            if starts_new:
                paragraphs.append(line)
            elif paragraphs[-1].endswith("-") and line[0].islower():
                paragraphs[-1] = paragraphs[-1][:-1] + line
            else:
                paragraphs[-1] += " " + line
    return paragraphs
```

PDF tiếng Việt gặp hai lỗi mà trang web không có. Thứ nhất, một số chữ bị tách đôi do phông chữ nhúng thiếu bảng ánh xạ ký tự. Chương trình nối hai mảnh liền nhau khi chuỗi ghép là một từ đã gặp trong các trang web cùng ngôn ngữ hoặc xuất hiện ít nhất 5 lần trong các PDF; cách này sửa được một phần nhưng vẫn sót, ví dụ "dinh dư ng" khi ký tự "ỡ" bị mất hẳn. Thứ hai, bảng biểu bị trích thành các cột xen kẽ nhau; lỗi này chưa được sửa. Bản PDF quét thành ảnh sẽ cần thêm bước nhận dạng ký tự (OCR), nhưng kho hiện tại không có tài liệu nào như vậy.

Chia đoạn. Văn bản sau làm sạch được chia thành đoạn khoảng 350 token, hai đoạn liền nhau gối 50 token, và chỉ cắt ở ranh giới đoạn văn. Số token được đếm bằng bộ tách từ của chính mô hình bge-m3. Đoạn gối nhau để một ý nằm vắt qua ranh giới vẫn xuất hiện trọn vẹn ở ít nhất một đoạn. Kết quả: 424 đoạn, trung bình 337 token, nhỏ nhất 63, lớn nhất 400. Mỗi đoạn mang mã tài liệu, thứ tự trong tài liệu, ngôn ngữ, và qua tài liệu là URL, đơn vị xuất bản và ngày tải; đây là thông tin hiển thị ở phần trích dẫn.

Kiểm tra 30 đoạn ngẫu nhiên (`results/chunk_audit.md`) không thấy đoạn nào là menu, quảng cáo hay đoạn rỗng.

5.4. Biểu diễn tri thức

5.4.1 Vector của đoạn văn

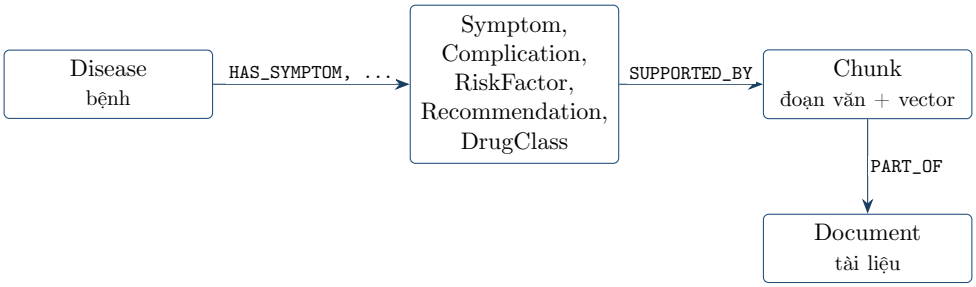
Biểu diễn số (embedding) chuyển một đoạn văn thành một vector. Hệ thống dùng mô hình `BAAI/bge-m3` [3], tạo vector 1.024 chiều. Mô hình được huấn luyện để câu hỏi và đoạn văn cùng nghĩa có vector gần nhau, kể cả khi khác ngôn ngữ, nên một câu hỏi tiếng Việt có thể tìm được đoạn tiếng Anh. Độ gần đo bằng cosine:

$$\cos(u, v) = \frac{\sum_i u_i v_i}{\sqrt{\sum_i u_i^2}\,\sqrt{\sum_i v_i^2}}.$$

Các vector được chuẩn hóa về độ dài 1, nên cosine bằng tích vô hướng.

5.4.2 Lược đồ đồ thị

Lược đồ quy định các loại nút, thuộc tính và quan hệ được phép, và nằm trong một tệp `backend/app/schema.py`. Chương trình nạp dữ liệu, chương trình truy xuất và kiểm thử cùng đọc tệp này, nên đổi lược đồ ở một chỗ là đủ.



Hình 5.2: Lược đồ tri thức y tế trong `KB_diabet`. Quan hệ `SUPPORTED_BY` mang thuộc tính `quote`: câu trích nguyên văn.

Bảng 5.4: Số nút theo nhãn

Nhãn	Số nút
Chunk	424
Complication	257
Disease	5
Document	35
DrugClass	191
Recommendation	840
RiskFactor	275
SurveyRecord	600
SurveyStatus	3
Symptom	216

Bảng 5.5: Số quan hệ theo loại

Quan hệ	Số lượng
`HAS_COMPLICATION`	277
`HAS_DRUG_CLASS`	208
`HAS_RECOMMENDATION`	868
`HAS_REPORTED_STATUS`	600
`HAS_RISK_FACTOR`	298
`HAS_SYMPTOM`	236
`PART_OF`	424
`SUPPORTED_BY`	2.116

Nút `SurveyRecord` và `SurveyStatus` là mẫu 600 dòng của khảo sát BRFSS, dùng cho bộ phân loại ở mục 4 của project; chúng không tham gia vào hỏi đáp.

5.4.3 Trích dữ kiện y tế có kiểm chứng

Đồ thị cần các nút dữ kiện, nhưng tài liệu chỉ có văn bản. Hệ thống dùng chính mô hình ngôn ngữ để trích dữ kiện, rồi kiểm kết quả bằng code.

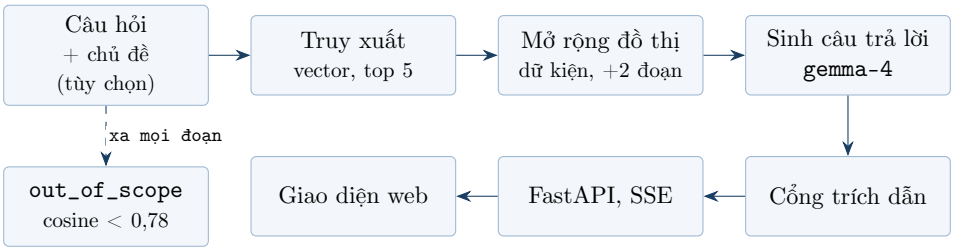
Mỗi đoạn văn được gửi cho mô hình `gemma-4` kèm yêu cầu liệt kê các dữ kiện mà đoạn nêu rõ, ở dạng JSON: loại dữ kiện, bệnh liên quan, tên ngắn tiếng Việt, điều kiện áp dụng nếu có, và một câu trích nguyên văn từ đoạn. Mô hình có thể bịa câu trích, nên chương trình kiểm: câu trích phải là chuỗi con của đoạn văn sau khi chuẩn hóa khoảng trắng và chữ hoa. Dữ kiện không qua bước kiểm bị loại và ghi vào `results/facts_rejected.jsonl`.

> Trên toàn kho, mô hình đề xuất 2.195 dữ kiện. Chương trình giữ 2.124, loại 60 dữ kiện có câu trích không nằm trong đoạn và 11 dữ kiện sai loại hoặc sai mã bệnh. Sau khi gộp các dữ kiện trùng tên, đồ thị còn 1.779 nút dữ kiện. Bước này tốn 522.949 token.

Dữ kiện trùng tên được gộp thành một nút, kể cả khi đến từ hai nguồn khác nhau; bảng đồng nghĩa `pipeline/synonyms.yaml` gộp các cách gọi như "khát nước nhiều" và "khát nhiều". Khi nhiều nguồn cùng nêu một khuyến nghị, nút khuyến nghị có nhiều cạnh `SUPPORTED_BY`. Ví dụ nút "giảm cân nếu thừa cân và duy trì cân nặng đó" được cả NIDDK và NDSS hỗ trợ.

Bước kiểm câu trích đảm bảo dữ kiện có trong nguồn, nhưng không kiểm được dữ kiện có được phân loại đúng hay không: cụm "không cần liệu pháp insulin" bị xếp vào triệu chứng của đái tháo đường típ 2. Việc gộp cũng còn yếu do tên ngắn không thống nhất giữa các lần gọi; chỉ 52 dữ kiện được nguồn của cả hai ngôn ngữ cùng hỗ trợ.

5.5. Pipeline RAG



Hình 5.3: Một lượt hỏi đáp. Câu hỏi quá xa mọi đoạn trong kho bị từ chối mà không gọi mô hình ngôn ngữ.

5.5.1 Truy xuất

Hệ thống cài đặt ba cách tìm đoạn văn:

- **BM25** [4]: chấm điểm theo số từ chung giữa câu hỏi và đoạn, có trọng số cho từ hiếm.
- **Vector**: lấy 20 đoạn có cosine cao nhất qua chỉ mục vector của Neo4j.
- **Đồ thị**: từ các đoạn tốt nhất, đi theo `SUPPORTED_BY` tới dữ kiện rồi đi tiếp tới các đoạn khác hỗ trợ cùng dữ kiện; nếu biết chủ đề, lấy thêm các đoạn hỗ trợ dữ kiện của bệnh tương ứng.

Khi dùng nhiều cách, các danh sách được gộp bằng RRF (Reciprocal Rank Fusion) [5]: đoạn ở hạng r của một danh sách nhận điểm $1/(k+r)$ với $k = 60$, và điểm của đoạn là tổng điểm qua các danh sách.

Mục 5.6 cho thấy xếp hạng chỉ bằng vector cho kết quả tốt nhất. Cấu hình chạy thật vì vậy xếp hạng bằng vector, lấy 5 đoạn đầu, rồi dùng đồ thị theo hai cách không làm xáo trộn thứ hạng: gắn các dữ kiện (kèm điều kiện áp dụng) của mỗi đoạn vào ngữ cảnh, và nối thêm tối đa 2 đoạn liên kết qua dữ kiện chung vào sau 5 đoạn đầu.

Đoạn mã dưới gồm truy vấn Cypher của nhánh đồ thị và phần chính của `retrieve`. Truy vấn Cypher có hai vế: vế đầu đi từ đoạn hạt giống qua `SUPPORTED_BY` tới dữ kiện rồi tới đoạn

khác, về sau lấy các đoạn hỗ trợ dữ kiện của bệnh ứng với chủ đề; đoạn được nhiều dữ kiện chung nối tới thì xếp trước. Trong `retrieve`, các đoạn mở rộng được nối sau `fused` với điểm 0, nên không đổi thứ hạng của k đoạn đầu.

Đoạn mã 5.2. Truy xuất: nhánh đồ thị và mở rộng sau xếp hạng. `ass_08/A8_02_TranXuanThanh_280/backend/app/retrieval.py`, dòng 99–107, 154–167.

```
            CALL {
              MATCH (seed:Chunk)<-[:SUPPORTED_BY]-(f)-[:SUPPORTED_BY]->(c:Chunk)
              WHERE seed.id IN $seeds AND NOT c.id IN $seeds
              RETURN c.id AS id, count(DISTINCT f) AS w
              UNION ALL
              MATCH (:Disease {id: $disease})-->(f)-[:SUPPORTED_BY]->(c:Chunk)
              RETURN c.id AS id, count(DISTINCT f) AS w
            }
            RETURN id, sum(w) AS weight ORDER BY weight DESC, id LIMIT $k
# ...
    vector = vector_search(embed([query])[0])
    rankings: dict[Branch, list[str]] = {}
    if "bm25" in branches:
        rankings["bm25"] = bm25_search(query)
    if "vector" in branches:
        rankings["vector"] = [cid for cid, _ in vector]
    if "graph" in branches:
        seeds = list(dict.fromkeys(rankings.get("vector", [])[:5] + rankings.get("bm25", [])[:5]))
        rankings["graph"] = graph_search(seeds, topic)
    fused = rrf(rankings)[:k]
    if expand:
        ranked = [cid for cid, _, _ in fused]
        extra = [cid for cid in graph_search(ranked[:3], topic) if cid not in ranked][:expand]
        fused += [(cid, 0.0, ["graph"]) for cid in extra]
```

5.5.2 Sinh câu trả lời

Các đoạn được đánh nhãn S1, S2, ... và gửi cho mô hình cùng câu hỏi. Lời nhắc yêu cầu mô hình:

1. chỉ dùng thông tin trong các nguồn được cung cấp;
2. đặt dấu `[[cite:Sn]]` sau mỗi ý;
3. giữ nguyên điều kiện áp dụng của khuyến nghị;
4. không thêm liều thuốc hay con số không có trong nguồn;
5. viết đúng một dòng `KHONG_DU_BANG_CHUNG` khi các nguồn không trả lời được câu hỏi.

Mô hình là `gemma-4`, gọi qua API tương thích OpenAI, nên có thể đổi sang mô hình khác mà không sửa mã.

5.5.3 Cổng trích dẫn

Lời nhắc chỉ là yêu cầu; mô hình không phải lúc nào cũng làm theo. Cổng trích dẫn là bước kiểm bằng code, chạy trước khi câu trả lời được hiển thị:

- chuẩn hóa mọi cách viết trích dẫn của mô hình (`[[S5]]`, `[S1]`, `[[1, 3]]`) về một dạng;
- bỏ các trích dẫn trỏ tới nguồn không được cung cấp;
- chuyển câu trả lời không còn trích dẫn hợp lệ nào thành trạng thái `insufficient_evidence`.

Ba bước này tương ứng ba dòng trong hàm `gate`: `CITE_VARIANTS.sub` chuẩn hóa cách viết, `CITE.sub` xóa trích dẫn có nhãn ngoài tập `valid`, và `if not used` trả về trạng thái thiếu bằng chứng.

Đoạn mã 5.3. Cổng trích dẫn. `ass_08/A8_02_TranXuanThanh_280/backend/app/answer.py`, dòng 14–17, 67–77.

```
CITE = re.compile(r"\[\[cite:(S\d+)\]\]")
ALIAS = r"(?:cite:\s*)?S?\d+"
CITE_VARIANTS = re.compile(rf"\[\[?\s*({ALIAS}(?:\s*,\s*{ALIAS})*)\s*\]?\]")
```

```
NO_EVIDENCE = "KHONG_DU_BANG_CHUNG"
# ...
def gate(raw: str, n_sources: int) -> Gated:
    """Drop citations to aliases that were never provided; no surviving citation means no answer."""
    valid = {alias(i) for i in range(n_sources)}
    raw = CITE_VARIANTS.sub(lambda m: "".join(f"[[cite:S{n}]]" for n in re.findall(r"\d+", m.group(1))), raw)
    if NO_EVIDENCE in raw:
        return Gated("", AnswerState.INSUFFICIENT_EVIDENCE, [])
    text = CITE.sub(lambda m: m.group(0) if m.group(1) in valid else "", raw).strip()
    used = list(dict.fromkeys(CITE.findall(text)))
    if not used:
        return Gated("", AnswerState.INSUFFICIENT_EVIDENCE, [])
    return Gated(text, AnswerState.ANSWERED, used)
```

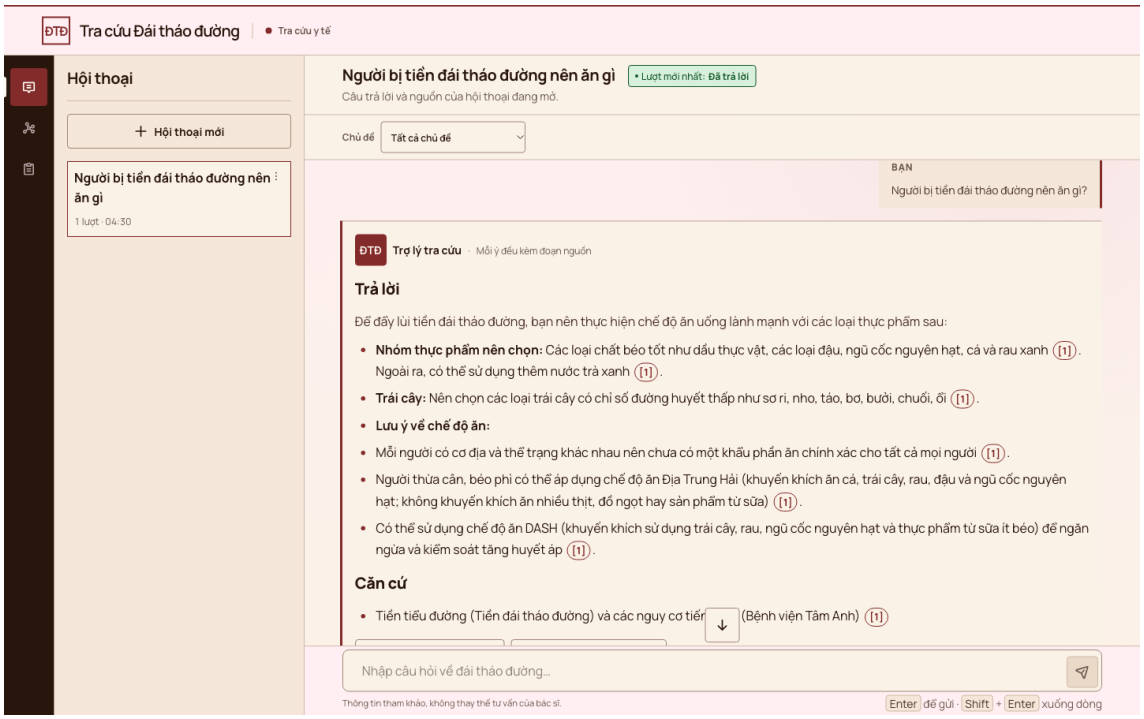
Bảng 5.6: Ba trạng thái câu trả lời

Trạng thái	Ý nghĩa
`answered`	Có ít nhất một trích dẫn hợp lệ; giao diện hiện câu trả lời và phần căn cứ.
`insufficient_evidence`	Câu hỏi trong lĩnh vực nhưng các nguồn tìm được không trả lời được.
`out_of_scope`	Câu hỏi không gần nội dung nào của kho; không gọi mô hình ngôn ngữ.

Vì cổng chạy trên toàn bộ câu trả lời, máy chủ đợi mô hình viết xong rồi mới gửi chữ cho trình duyệt. Đổi lại, người dùng không bao giờ thấy một ý chưa qua kiểm tra.

5.5.4 Phục vụ

Máy chủ FastAPI gửi kết quả qua SSE (Server-Sent Events), cách máy chủ gửi dần từng sự kiện trên một kết nối HTTP. Một lượt hỏi phát lần lượt `stream.started`, `sources.snapshot` (các đoạn đã tìm được), `answer.started`, các sự kiện `answer.text.delta` và `answer.citation`, `answer.final` và `stream.completed`. Giao diện Next.js hiển thị câu trả lời, trích dẫn đánh số và phần căn cứ ghi tài liệu, đơn vị xuất bản.



Hình 5.4: Giao diện hỏi đáp: câu trả lời có trích dẫn đánh số, phần căn cứ ghi tài liệu và đơn vị xuất bản.

5.6. Đánh giá

5.6.1 Phương pháp

Bộ câu hỏi. Bộ đánh giá gồm 30 câu: 26 câu trong phạm vi (khái niệm, triệu chứng, nguy cơ, chẩn đoán, ăn uống, vận động, thuốc, biến chứng, tổng hợp; trong đó 4 câu tiếng Anh) và 4 câu ngoài phạm vi, gồm hai câu gần lĩnh vực (ung thư tuyến tụy, thuốc cảm cúm).

Tập tham chiếu. Với mỗi câu, chương trình gộp 20 đoạn đầu của mọi cách truy xuất (trung bình 48 đoạn), và mô hình ngôn ngữ chấm từng đoạn có chứa câu trả lời trực tiếp hay không. Bộ chấm phải trích nguyên văn câu trả lời trong đoạn, và chương trình kiểm câu trích có thật. Kết quả là 269 cặp câu hỏi–đoạn liên quan. Đọc lại 12/15 nhãn dương ngẫu nhiên thấy khớp với đánh giá của người đọc.

Chỉ số truy xuất. Với R là tập đoạn liên quan và T_k là k đoạn đầu hệ thống trả về:

$$\text{Hit@}k = \mathbb{1}[\,|R \cap T_k| > 0\,], \qquad \text{Precision@}k = \frac{|R \cap T_k|}{k},$$

$$\text{Recall@}k = \frac{|R \cap T_k|}{\min(|R|, k)}, \qquad \text{MRR} = \frac{1}{\text{hạng của đoạn liên quan đầu tiên}}.$$

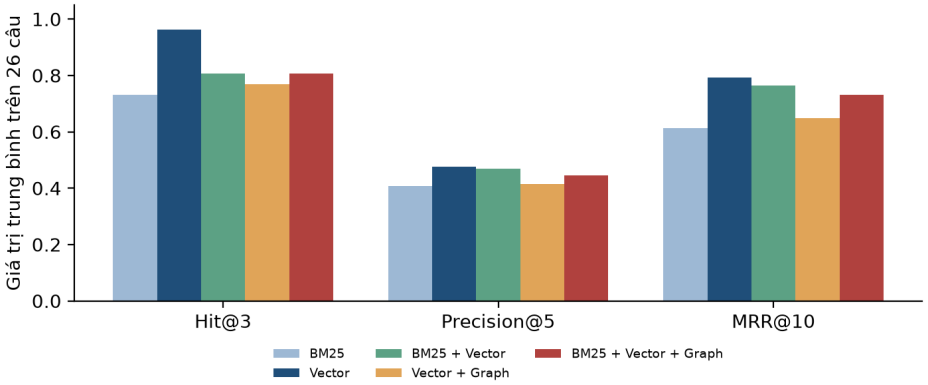
Recall@k chia cho $\min(|R|, k)$: một câu có 20 đoạn liên quan thì 5 kết quả đầu không thể chứa đủ 20, và chia cho 20 sẽ phạt hệ thống vì điều nó không làm được.

Chất lượng câu trả lời. Bộ chấm tách câu trả lời thành các phát biểu đơn và xét từng phát biểu với toàn văn các đoạn nó trích: được hỗ trợ, hỗ trợ một phần, hoặc không được hỗ trợ.

5.6.2 So sánh các cách truy xuất

Bảng 5.7: Truy xuất trên 26 câu trong phạm vi, xếp hạng 10 đoạn, không mở rộng

Cấu hình	Hit@3	Hit@5	Precision@5	Recall@5	MRR@10
BM25	0,731	0,846	0,408	0,425	0,613
Vector	0,962	1,000	0,477	0,498	0,792
BM25 + Vector	0,808	0,923	0,469	0,486	0,765
Vector + Graph	0,769	0,923	0,415	0,435	0,648
BM25 + Vector + Graph	0,808	0,923	0,446	0,464	0,731



Hình 5.5: Ba chỉ số truy xuất theo cấu hình.

Vector đứng đầu ở mọi chỉ số; 5 đoạn đầu chứa ít nhất một đoạn liên quan ở cả 26 câu. Thêm BM25 làm kết quả kém đi: BM25 tách tiếng Việt theo âm tiết, và các âm tiết "đái", "tháo", "đường" xuất hiện ở hầu hết mọi đoạn nên đóng góp nhiều nhiễu. Thêm đồ thị vào RRF như một danh sách ngang hàng cũng làm kết quả kém đi: đồ thị trả về các đoạn hỗ trợ cùng dữ kiện, nhưng "cùng dữ kiện" không có nghĩa là "trả lời cùng câu hỏi", và các đoạn này đẩy đoạn tốt của vector xuống dưới.

5.6.3 Đóng góp của đồ thị

Khi dùng đồ thị để nối thêm tối đa 2 đoạn sau 5 đoạn đầu, trên 26 câu đồ thị thêm 51 đoạn; 17,6% trong số đó liên quan đến câu hỏi, 39,2% là đoạn ở ngôn ngữ còn lại, và 7 đoạn liên quan nằm ngoài 10 đoạn đầu của vector.

Bảng 5.8: Câu trả lời có và không có mở rộng bằng đồ thị

Chỉ số	Không mở rộng	Mở rộng 2 đoạn
Câu trong phạm vi được trả lời	24/26	25/26
Số phát biểu được chấm	175	186
Phát biểu được nguồn hỗ trợ	98,9%	97,9%
Phát biểu không được hỗ trợ	0,6%	0,0%
Từ chối nhầm câu trong phạm vi	7,7%	3,9%

Hai cấu hình chênh nhau một câu hỏi trên một lần chạy, nên thí nghiệm không cho thấy đồ thị cải thiện câu trả lời một cách đo được. Mở rộng không làm câu trả lời kém đi và đưa

thêm bằng chứng từ ngôn ngữ còn lại, nên cấu hình chạy thật giữ bước này.

5.6.4 Chất lượng câu trả lời

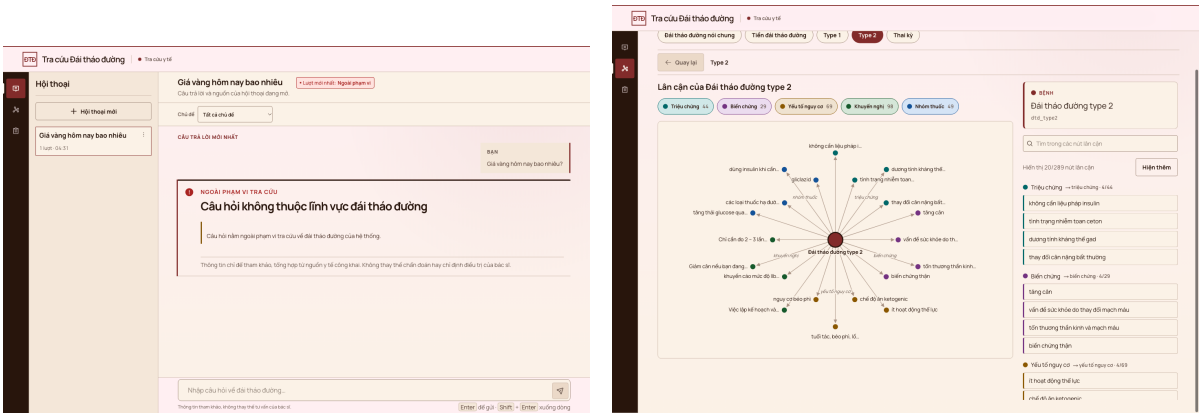
25/26	97,9%	100%	2,46 s
câu trong phạm vi được trả lời	phát biểu được nguồn hỗ trợ	câu ngoài phạm vi bị từ chối	trung vị thời gian một câu

Thời gian lâu nhất là 8,82 giây, phần lớn là thời gian mô hình viết câu trả lời.

Các lỗi tìm ra trong quá trình đo. Số liệu trên là kết quả sau ba vòng đo và sửa. Hai vòng đầu lộ ra lỗi của hệ thống đo và của cổng trích dẫn, không phải của mô hình:

1. Lần đo đầu cho 39,1% phát biểu không được hỗ trợ. Nguyên nhân: bộ chấm chỉ nhận 600 ký tự đầu của mỗi nguồn. Sau khi cho bộ chấm đọc toàn văn, tỷ lệ này còn 4,0%.
2. Tỷ lệ từ chối nhầm câu trong phạm vi khi đó là 19,2%. Đọc đầu ra thô cho thấy mô hình viết trích dẫn dạng `[[S5]]`, `[[1, 3, 5]]`, và cổng trích dẫn xóa hết vì không khớp dạng `[[cite:S5]]`. Sau khi cổng chuẩn hóa các cách viết này, tỷ lệ từ chối nhầm còn 3,9%.

Cả hai lỗi chỉ lộ ra khi đọc đầu ra thô thay vì tin con số tổng hợp.



Hình 5.6: Câu hỏi ngoài phạm vi bị từ chối (trái) và trang đồ thị tri thức quanh nút đái tháo đường típ 2 (phải).

5.7. Hạn chế và vai trò trong hệ đa agent

Hạn chế.

- Bộ đánh giá nhỏ (30 câu), tập tham chiếu do mô hình ngôn ngữ gán; khoảng 12/15 nhãn đúng khi đọc lại. Chênh lệch một câu hỏi giữa các cấu hình nằm trong mức dao động của một lần chạy.
- PDF tiếng Việt còn lỗi tách chữ và lỗi bảng; chưa có OCR cho PDF quét.
- Gộp dữ kiện còn yếu, một số dữ kiện bị gán sai loại.
- Chữ chỉ hiện khi mô hình viết xong, vì cổng trích dẫn chạy trên toàn bộ câu trả lời.
- Hệ thống chưa được chuyên gia y tế đánh giá.

Vai trò trong hệ đa agent. Pipeline RAG trong chương này trả lời từng câu hỏi độc lập: nó không hỏi lại người dùng, không nhớ hồ sơ sức khỏe, và không kết hợp tri thức với kết quả của mô hình dự đoán nguy cơ. Chương 6 đóng gói pipeline thành một công cụ `search_knowledge` với đầu vào là câu hỏi và chủ đề, đầu ra là câu trả lời, trạng thái và danh sách trích dẫn. Agent

tra cứu tri thức gọi công cụ này; agent tư vấn dùng các trích dẫn nó trả về làm căn cứ cho lời khuyên.

Chương 6

Agent và hệ đa agent

Hệ thống RAG ở Chương 5 trả lời một câu hỏi rồi dừng. Nó không tự quyết định cần làm gì tiếp theo: không hỏi lại khi thiếu thông tin, không gọi mô hình dự đoán nguy cơ, không ghép kết quả của nhiều bước thành một lời khuyên. Chương này trình bày khái niệm agent, tức chương trình tự chọn hành động để đạt mục tiêu, và hệ đa agent, tức nhiều agent chuyên môn phối hợp với nhau. Phần cuối chương trình bày Diabeque, hệ đa agent hỗ trợ sàng lọc và tư vấn bệnh đái tháo đường mà nhóm đã cài đặt bằng LangGraph, cùng kết quả đánh giá so với một agent duy nhất.

6.1. Agent

6.1.1 Định nghĩa

Theo Russell và Norvig [9], agent là thực thể *cảm nhận* môi trường qua cảm biến và *tác động* lên môi trường qua cơ cấu chấp hành. Hành vi của agent được mô tả bằng hàm agent, ánh xạ chuỗi cảm nhận đã có thành hành động. Agent hợp lý là agent chọn hành động có kỳ vọng tối đa hóa thước đo hiệu năng, dựa trên những gì nó cảm nhận và biết.

Một bài toán agent được mô tả bằng bốn yếu tố PEAS: thước đo hiệu năng (Performance), môi trường (Environment), cơ cấu chấp hành (Actuators) và cảm biến (Sensors).

Bảng 6.1: Mô tả PEAS của trợ lý sàng lọc và tư vấn đái tháo đường

Yếu tố	Nội dung
Thước đo hiệu năng	Thu đủ thông tin cần thiết với ít câu hỏi; ước lượng nguy cơ có hiệu chỉnh; lời khuyên đúng và có nguồn; không đưa chẩn đoán khẳng định hay liều thuốc; chuyển người dùng tới cơ sở y tế khi có dấu hiệu khẩn cấp.
Môi trường	Cuộc hội thoại với người dùng; kho tri thức `KB_diabet`; mô hình phân loại BRFSS.
Cơ cấu chấp hành	Gửi câu hỏi hoặc câu trả lời cho người dùng; gọi công cụ (tra cứu, dự đoán, tính BMI).
Cảm biến	Tin nhắn của người dùng; kết quả trả về từ công cụ.

6.1.2 Các loại agent cổ điển

Russell và Norvig chia agent thành năm loại theo mức độ phức tạp:

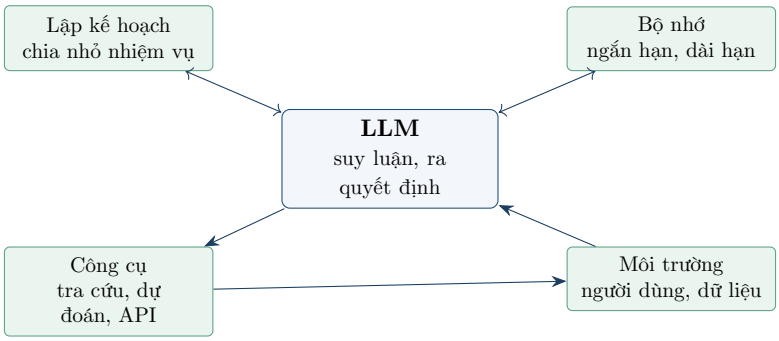
- **Phản xạ đơn giản**: hành động theo luật *nếu–thì* trên cảm nhận hiện tại.
- **Phản xạ dựa trên mô hình**: lưu trạng thái nội bộ về phần môi trường không quan sát được.
- **Dựa trên mục tiêu**: chọn chuỗi hành động dẫn tới trạng thái đích.
- **Dựa trên tiện ích**: so sánh các trạng thái bằng hàm tiện ích khi có nhiều cách đạt mục tiêu.
- **Học**: cải thiện hành vi qua phản hồi.

Bộ phân loại ở các chương trước gần với agent phản xạ đơn giản: nhận một vector đặc trưng và trả một nhãn. Pipeline RAG có thêm trạng thái (các đoạn đã tìm được) nhưng vẫn đi theo

một trình tự cố định. Trợ lý tư vấn cần ít nhất mức dựa trên mục tiêu: mục tiêu là đưa ra lời khuyên có căn cứ, và để đạt được nó agent phải tự quyết định khi nào hỏi thêm, khi nào tra cứu, khi nào dự đoán.

6.1.3 Agent dựa trên mô hình ngôn ngữ lớn

Từ năm 2022, mô hình ngôn ngữ lớn được dùng làm bộ phận ra quyết định của agent [15]. Một LLM agent gồm bốn thành phần:



Hình 6.1: Bốn thành phần của một LLM agent.

Lập kế hoạch. Chain-of-thought [11] yêu cầu mô hình viết các bước suy luận trung gian trước khi trả lời; cách này giúp mô hình chia một nhiệm vụ phức tạp thành các bước nhỏ. Reflexion [14] cho agent tự nhận xét kết quả của lần thử trước và rút kinh nghiệm cho lần sau.

Công cụ. Mô hình ngôn ngữ chỉ sinh văn bản. Để tác động lên thế giới, mô hình sinh một lời gọi công cụ có cấu trúc, chương trình thực thi lời gọi đó và trả kết quả về cho mô hình [13]. Các API hiện nay mô tả công cụ bằng tên, mô tả và lược đồ JSON của tham số. Ví dụ công cụ dự đoán nguy cơ:

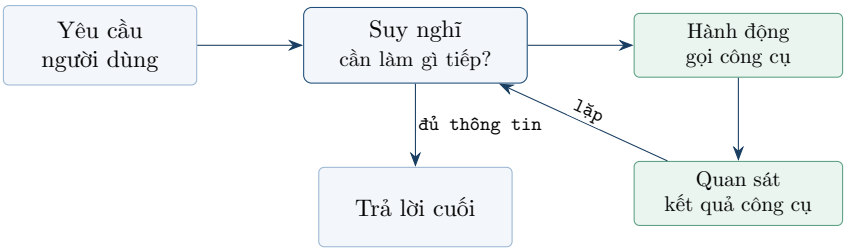
```
{
  "name": "predict_diabetes_risk",
  "description": "Ước lượng xác suất ba nhóm tình trạng từ 19 đặc trưng BRFSS.",
  "parameters": {
    "type": "object",
    "properties": {
      "BMI":      {"type": "number", "minimum": 12, "maximum": 98},
      "HighBP":   {"type": "integer", "enum": [0, 1]},
      "Age":      {"type": "integer", "minimum": 1, "maximum": 13}
    },
    "required": ["BMI", "HighBP", "Age"]
  }
}
```

Lược đồ có hai tác dụng: mô hình biết công cụ cần gì, và chương trình kiểm được tham số trước khi thực thi.

Bộ nhớ. Bộ nhớ ngắn hạn là lịch sử hội thoại và trạng thái của phiên hiện tại, nằm trong ngữ cảnh của mô hình. Bộ nhớ dài hạn nằm ngoài mô hình và được truy xuất khi cần; KB ở Chương 5 là một dạng bộ nhớ dài hạn.

6.1.4 Vòng lặp ReAct

ReAct [12] xen kẽ suy luận và hành động trong một vòng lặp. Ở mỗi bước, mô hình viết một suy nghĩ (Thought), chọn một hành động (Action, thường là gọi công cụ), rồi nhận kết quả quan sát (Observation). Vòng lặp dừng khi mô hình chọn hành động trả lời cuối cùng hoặc khi vượt số bước tối đa.



Hình 6.2: Vòng lặp ReAct.

Ví dụ một vết chạy của trợ lý khi người dùng hỏi "Tôi cao 1m65, nặng 78kg, huyết áp cao, có nguy cơ tiểu đường không?":

Suy nghĩ: Người dùng cho chiều cao, cân nặng và huyết áp; thiếu tuổi, mức vận động, cholesterol.
Hành động: `compute_bmi(height_m=1.65, weight_kg=78)`
Quan sát: `{"bmi": 28.7}`
Suy nghĩ: Cần hỏi thêm nhóm tuổi trước khi dự đoán.
Hành động: hỏi người dùng "Anh/chị bao nhiêu tuổi?"
...
Hành động: `predict_diabetes_risk({...})`
Quan sát: `{"no": 0.58, "pre": 0.07, "diabetes": 0.35}`
Hành động: `search_knowledge("giảm nguy cơ đái tháo đường khi thừa cân", topic="prevention")`

6.1.5 Workflow và agent

Không phải bài toán nào cũng cần agent tự quyết định. Anthropic [18] phân biệt *workflow*, trong đó các bước và thứ tự gọi mô hình được viết sẵn trong mã, và *agent*, trong đó mô hình tự chọn bước tiếp theo. Workflow dễ dự đoán, dễ kiểm thử, rẻ hơn; agent linh hoạt hơn nhưng khó kiểm soát và tốn nhiều lần gọi mô hình hơn. Lời khuyên thực hành là bắt đầu bằng cách đơn giản nhất chạy được, và chỉ thêm quyền tự quyết khi workflow không đủ.

Pipeline RAG ở Chương 5 là một workflow. Trong Diabeque (mục 6.3), điều phối viên dùng LLM để chọn nhánh vì thứ tự hỏi–tra cứu–dự đoán phụ thuộc vào người dùng; các nhánh và điều kiện chuyển giữa các agent lại là đồ thị viết sẵn trong code, nên hệ thống nằm giữa workflow và agent.

6.2. Hệ đa agent

6.2.1 Khái niệm

Hệ đa agent (multi-agent system) là hệ gồm nhiều agent tương tác với nhau trong cùng một môi trường [10]. Mỗi agent có mục tiêu, thông tin và khả năng riêng; hệ đạt mục tiêu chung qua hợp tác, phối hợp hoặc thương lượng. Trong hệ đa agent dựa trên LLM, mỗi agent thường là một lời nhắc hệ thống riêng, một tập công cụ riêng, chạy trên cùng hoặc khác mô hình.

6.2.2 Vì sao chia thành nhiều agent

- **Chuyên môn hóa.** Mỗi agent có lời nhắc ngắn, tập trung vào một việc, và chỉ thấy công cụ nó cần. Một lời nhắc chứa mọi quy tắc của mọi việc dễ làm mô hình bỏ sót quy tắc.
- **Kiểm thử từng phần.** Agent thu thập thông tin có thể được kiểm thử riêng bằng bộ hội thoại mẫu, không phụ thuộc vào chất lượng tra cứu.
- **Tách quyền.** Agent an toàn có quyền chặn câu trả lời của agent khác; quyền này không nên nằm trong cùng agent sinh câu trả lời.

- **Thay thế độc lập.** Có thể đổi mô hình dự đoán hoặc kho tri thức mà không sửa phần còn lại.

Đổi lại, nhiều agent nghĩa là nhiều lần gọi mô hình, độ trễ cao hơn, và lỗi của một agent có thể lan sang agent khác.

6.2.3 Các mẫu kiến trúc

Bảng 6.2: Các mẫu tổ chức hệ đa agent

Mẫu	Cách hoạt động	Phù hợp khi
Tuần tự (pipeline)	Đầu ra của agent này là đầu vào của agent sau, thứ tự cố định.	Các bước luôn giống nhau.
Định tuyến (router)	Một agent phân loại yêu cầu rồi chuyển cho đúng agent chuyên môn.	Yêu cầu thuộc các loại tách biệt.
Điều phối–chuyên gia (supervisor)	Agent điều phối giao việc, nhận kết quả, quyết định bước tiếp, gộp câu trả lời.	Một yêu cầu cần nhiều chuyên môn, thứ tự không cố định.
Phân cấp	Điều phối viên quản lý các nhóm, mỗi nhóm có điều phối viên riêng.	Hệ lớn, nhiều miền.
Tranh luận / phản biện	Nhiều agent đề xuất và phê bình lẫn nhau, hoặc một agent kiểm duyệt kết quả của agent khác [17].	Cần giảm sai sót, câu trả lời có rủi ro.

6.2.4 Giao tiếp và trạng thái chung

Agent trao đổi thông tin theo hai cách. Cách thứ nhất là **truyền thông điệp**: agent gửi tin nhắn cho nhau như một cuộc hội thoại, ví dụ AutoGen [16]. Cách thứ hai là **trạng thái chung** (blackboard): mọi agent đọc và ghi vào một đối tượng trạng thái có cấu trúc, ví dụ LangGraph [19]. Trạng thái chung có lược đồ rõ ràng, nên chương trình kiểm được agent nào đã ghi gì; truyền thông điệp tự do hơn nhưng khó kiểm hơn.

6.2.5 Framework

Bảng 6.3: Một số framework xây hệ đa agent

Framework	Mô hình tổ chức	Ghi chú
LangGraph	Đồ thị trạng thái: nút là agent hoặc hàm, cạnh có điều kiện, trạng thái chung có kiểu.	Kiểm soát luồng chặt, lưu và khôi phục trạng thái.
AutoGen	Các agent hội thoại với nhau; có agent đại diện người dùng.	Linh hoạt, khó dự đoán số lượt.
CrewAI	Nhóm agent có vai trò, nhiệm vụ; chạy tuần tự hoặc phân cấp.	Khai báo nhanh, ít kiểm soát chi tiết.
Tự viết	Vòng lặp gọi API có tool calling, trạng thái là một đối tượng Python.	Không phụ thuộc thư viện; phải tự viết lưu trạng thái.

Diabeque dùng LangGraph vì luồng hội thoại có các nhánh rõ ràng (khẩn cấp, thiếu thông tin, đủ thông tin), cần kiểm được trạng thái sau mỗi bước, và cần giữ hồ sơ người dùng qua nhiều lượt; LangGraph lưu trạng thái theo từng cuộc hội thoại bằng checkpointer.

6.3. Diabeque: hệ đa agent cho đái tháo đường

6.3.1 Phạm vi: sàng lọc, không chẩn đoán

Chẩn đoán đái tháo đường dựa trên xét nghiệm: đường huyết lúc đói, nghiệm pháp dung nạp glucose, HbA1c hoặc đường huyết bất kỳ kèm triệu chứng [8]. Dữ liệu BRFSS là câu trả lời khảo sát tự báo cáo, không có xét nghiệm. Bộ phân loại MLP của A8 nhận đúng chưa tới một phần tư số mẫu tiền đái tháo đường (524 trên 2.295, precision 0,0357), và các mạng học sâu của project cũng chỉ đạt macro-F1 khoảng 0,43 ở bài ba nhóm (Chương 2). Hệ thống vì vậy chỉ làm ba việc:

1. **ước lượng nguy cơ** từ thông tin người dùng cung cấp, kèm xác suất đã hiệu chỉnh;
2. **giải thích** kết quả và trả lời câu hỏi bằng tri thức có nguồn;
3. **tư vấn** lối sống và khuyên đi xét nghiệm khi nguy cơ cao.

Hệ thống không đưa ra kết luận "anh/chị bị đái tháo đường", không kê thuốc hay liều, và không thay thế bác sĩ.

6.3.2 Các agent

Hệ thống có bảy nút trong đồ thị LangGraph. Nguyên tắc phân chia: LLM chỉ làm ba việc cần hiểu ngôn ngữ (xác định ý định, trích thông tin từ câu nói tự do, viết câu trả lời từ nguồn); mọi việc có đáp án xác định được viết bằng code.

Bảng 6.4: Vai trò, cách ra quyết định và công cụ của từng agent

Agent	Việc làm	Cách làm
An toàn (vào)	Phát hiện dấu hiệu cấp cứu: lú lẫn, bất tỉnh, khó thở, nôn nhiều, đường huyết ≥ 400 mg/dL hoặc < 70 mg/dL.	Luật (biểu thức chính quy)
Điều phối	Xếp tin nhắn vào bốn ý định: `knowledge`, `assess_risk`, `greeting`, `out_of_scope`.	LLM gọi công cụ `IntentDecision`; luật dự phòng khi LLM lỗi
Thu thập thông tin	Trích thông tin sức khỏe từ lời nói, đổi sang mã BRFSS; hỏi tối đa 3 câu mỗi lượt cho 7 trường bắt buộc (tuổi, giới, BMI, huyết áp cao, mỡ máu cao, vận động, sức khỏe chung).	LLM `IntakeExtraction` + luật; `compute_bmi`, `validate_profile`
Dự đoán nguy cơ	Gọi mô hình ba nhóm (CNN-1D), hoặc mô hình Type 1/2 (LSTM) nếu người dùng nói đã được chẩn đoán; trường chưa hỏi được điền bằng trung vị và câu trả lời nêu rõ điều đó.	`predict_diabetes_risk`, `predict_diabetes_type`; văn bản sinh bằng code
Tra cứu tri thức	Trả lời câu hỏi kiến thức có trích dẫn.	`search_knowledge` (pipeline RAG, Chương 5)
Tư vấn	Dựng truy vấn từ các yếu tố nguy cơ trong hồ sơ, lấy khuyến nghị có nguồn, viết 3–6 lời khuyên.	`get_recommendations` + LLM + cổng trích dẫn
An toàn (ra)	Bỏ câu chứa liều thuốc hoặc kết luận chẩn đoán cá nhân; phần không còn trích dẫn bị thay bằng lời khuyên hỏi bác sĩ.	Luật

Thu thập: LLM và luật bổ sung cho nhau. Agent thu thập chạy hai bộ trích xuất trên cùng tin nhắn: LLM điền một lược đồ có kiểu, và một bộ luật nhận các cách nói phổ biến ("52 tuổi", "1m65", "80kg", "huyết áp bình thường"). Kết quả của LLM được ưu tiên; luật chỉ điền các trường LLM để trống. Việc đổi tuổi sang nhóm tuổi BRFSS (18–24 là nhóm 1, mỗi nhóm 5 năm, 80 trở lên là nhóm 13) và tính BMI làm bằng code, không giao cho LLM.

Đoạn mã 6.1. Agent thu thập: gộp kết quả LLM với luật rồi kiểm hồ sơ. `tieuluan_final/project/backend/app/agents/nodes.py`, dòng 247–248, 276–285.

```
def merge(llm: IntakeExtraction, rules: IntakeExtraction) -> IntakeExtraction:
    return llm.model_copy(update={k: v for k, v in rules.model_dump().items() if v is not None and getattr(llm, k)
    is None})
# ...
    try:
        x = structured(IntakeExtraction).invoke(prompt)
    except Exception as e:
        x, note = IntakeExtraction(), f"trích xuất lỗi ({type(e).__name__})"
    else:
        note = ""
    x = merge(x, rule_extract(state["message"]))
    feats, calls = to_features(x)
    checked = tools.validate_profile(feats)
    calls.append(call("validate_profile", {"profile": feats}, checked))
```

Hàm `merge` chỉ lấy giá trị của luật cho các trường LLM để trống. Khi LLM lỗi, `x` là hồ sơ rỗng và luật vẫn chạy, nên agent vẫn dùng được. `to_features` đổi tuổi, giới và chiều cao, cân nặng

sang mã BRFSS bằng code, và `validate_profile` kiểm khoảng giá trị trước khi hồ sơ được lưu vào trạng thái.

Dự đoán: hiệu chỉnh xác suất theo tần suất thật. Mô hình được huấn luyện với trọng số lớp $w_c \propto 1/n_c$ nên xác suất đầu ra bị đẩy về phía lớp hiếm. Nhân xác suất với n_c rồi chuẩn hóa đưa về tần suất của dữ liệu: $\tilde{p}_c = p_c\, n_c / \sum_k p_k\, n_k$. Giao diện hiển thị $\tilde{p}$ ("tỷ lệ ở những người có hồ sơ tương tự"). Ví dụ hồ sơ nam 52 tuổi, BMI 29,4, huyết áp cao, ít vận động: mô hình có trọng số cho 57% thuộc nhóm "đã được thông báo mắc đái tháo đường", sau hiệu chỉnh còn 35%. Công thức này nằm trong công cụ dự đoán; `class_counts` đọc số mẫu mỗi lớp từ cơ sở dữ liệu khảo sát.

Đoạn mã 6.2. Hiệu chỉnh xác suất về tần suất của dữ liệu. `tieuluan_final/project/backend/app/agents/tools.py`, dòng 58–66.

```
def prior_corrected(task: Task, probs: dict[str, float]) -> dict[str, float] | None:
    # ...
    counts = class_counts(task)
    if not counts:
        return None
    raw = {k: p * counts.get(k, 0) for k, p in probs.items()}
    total = sum(raw.values())
    return {k: round(v / total, 4) for k, v in raw.items()}
```

Vì sao agent an toàn đứng riêng. Nếu quy tắc an toàn chỉ nằm trong lời nhắc của agent tư vấn, việc tuân thủ phụ thuộc vào mô hình. Hai nút an toàn được viết bằng code (giống cổng trích dẫn ở Chương 5): mọi nhánh đều kết thúc ở An toàn (ra), và khi An toàn (vào) phát hiện khẩn cấp, luồng bỏ qua mọi agent khác và khuyên đến cơ sở y tế ngay.

Bộ lọc đầu ra làm việc theo từng câu. Câu chứa liều thuốc (số kèm đơn vị, trừ mg/dL, hoặc dạng chia liều như "2/3 tổng liều") bị bỏ. Câu kết luận chẩn đoán cá nhân bị bỏ, trừ khi là câu điều kiện hoặc câu hỏi.

Đoạn mã 6.3. Bộ lọc liều thuốc và chẩn đoán. `tieuluan_final/project/backend/app/agents/safety.py`, dòng 34–36, 52–63, 68.

```
DOSAGE = re.compile(r"\b\d+([.,]\d+)?\s*(mg|mcg|µg|g|ui|iu|đơn vị|ml)\b(?!\s*/\s*dl)", re.I)
# Dose split or weight-based dose without a unit: "2/3 tổng liều", "0,5 đơn vị/kg" is caught above.
DOSE_SPLIT = re.compile(r"\b\d+\s*/\s*\d+\s*(tổng\s+)?liều", re.I)
# ...
def filter_text(text: str) -> tuple[str, list[str]]:
    """Drop sentences with a dose or a definitive diagnosis. Returns cleaned text and the flags raised."""
    flags, lines = [], []
    for line in text.split("\n"):
        kept = []
        for sentence in SENTENCE.split(line):
            if DOSAGE.search(sentence) or DOSE_SPLIT.search(sentence):
                flags.append("dosage_removed")
            elif DIAGNOSIS.search(sentence) and not CONDITIONAL.search(sentence) and not
            sentence.rstrip().endswith("?"):
                flags.append("diagnosis_removed")
            elif sentence.strip():
                kept.append(sentence.strip())
        # ...
    return "\n".join(lines).strip(), sorted(set(flags))
```

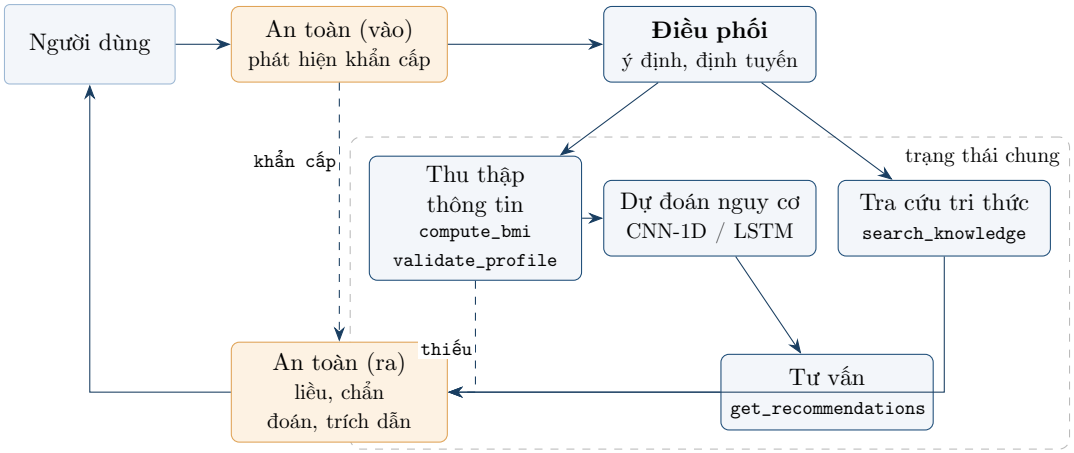
Nút An toàn (ra) chạy bộ lọc trên các phần do LLM viết. Phần nào không còn trích dẫn hợp lệ sau khi lọc bị thay bằng lời khuyên hỏi bác sĩ; nếu có câu liều thuốc bị bỏ, nút thêm một thông báo cố định giải thích vì sao hệ thống không đưa liều.

Đoạn mã 6.4. Nút An toàn (ra). `tieuluan_final/project/backend/app/agents/nodes.py`, dòng 422–432.

```
    for s in state.get("sections", []):
        if s["kind"] in FILTERED:
```

```
            text, raised = safety.filter_text(s["text"])
            flags += raised
            if not gen.CITE.search(text):
                flags.append("no_citation_left")
                text = "Nội dung bị loại ở bước kiểm an toàn. Anh/chị nên hỏi trực tiếp bác sĩ về vấn đề này."
            s = {**s, "text": text}
        sections.append(s)
    if "dosage_removed" in flags:
        sections.append({"kind": "notice", "title": "Về liều thuốc", "text": safety.DOSAGE_NOTICE})
```

6.3.3 Kiến trúc



Hình 6.3: Kiến trúc Diabeque theo mẫu điều phối–chuyên gia, có hai lớp kiểm an toàn. Mũi tên nét đứt là các nhánh dừng sớm.

Một lượt đi theo một trong ba nhánh:

- **Kiến thức**: điều phối → tra cứu tri thức → an toàn (ra).
- **Đánh giá nguy cơ**: điều phối → thu thập. Nếu hồ sơ còn thiếu, hệ thống hỏi thêm và kết thúc lượt; lượt sau người dùng trả lời, thu thập cập nhật hồ sơ. Khi đủ: dự đoán → tư vấn → an toàn (ra).
- **Khẩn cấp**: an toàn (vào) → an toàn (ra).

Chào hỏi và câu ngoài phạm vi được điều phối viên trả lời bằng văn bản cố định rồi đi thẳng tới an toàn (ra).

6.3.4 Trạng thái chung và cài đặt bằng LangGraph

Mọi agent đọc và ghi một đối tượng `SessionState`. Phần lưu qua các lượt gồm hồ sơ, cờ "đã được chẩn đoán", cờ "đang chờ trả lời câu hỏi thu thập", các trường đã hỏi và kết quả dự đoán. Phần làm mới mỗi lượt gồm tin nhắn, ý định, các trường còn thiếu, nguồn đã truy xuất, các phần câu trả lời, cờ an toàn và vết chạy.

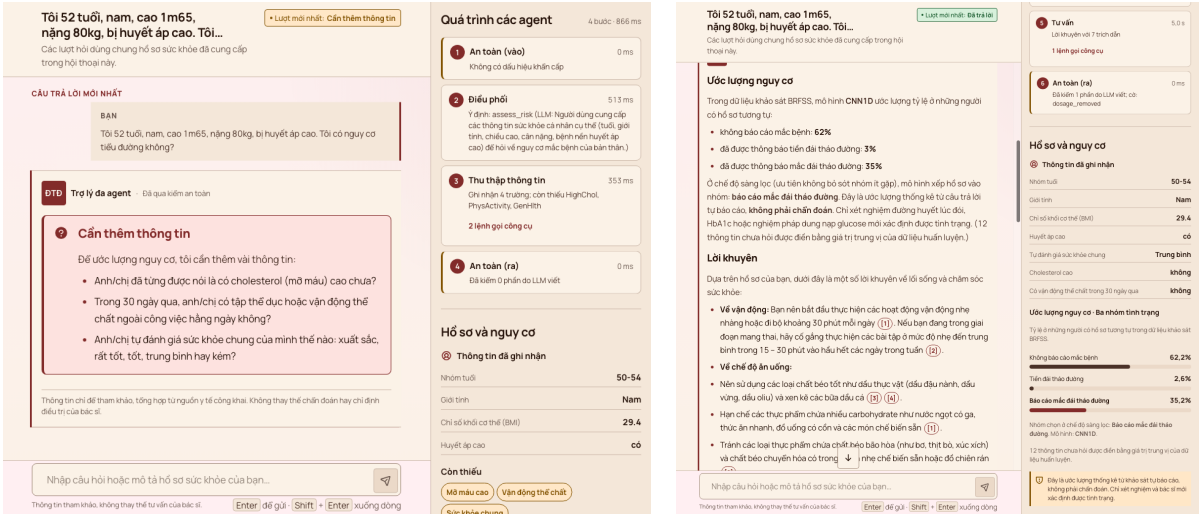
```
g = StateGraph(SessionState)
g.add_node("safety_in", nodes.safety_in)        # ... 7 nút
g.add_edge(START, "safety_in")
g.add_conditional_edges("safety_in",    after_safety_in,    ["orchestrator", "safety_out"])
g.add_conditional_edges("orchestrator", after_orchestrator, ["knowledge", "intake", "safety_out"])
g.add_conditional_edges("intake",       after_intake,       ["risk", "safety_out"])
g.add_edge("risk", "advisor");  g.add_edge("advisor", "safety_out");  g.add_edge("knowledge", "safety_out")
app = g.compile(checkpointer=InMemorySaver())   # trạng thái lưu theo conversation_id
```

Các cạnh có điều kiện là hàm thuần trên trạng thái, ví dụ `after_intake` trả về `"risk"` khi danh sách trường còn thiếu rỗng. Đồ thị không có chu trình trong một lượt, nên không thể xảy ra

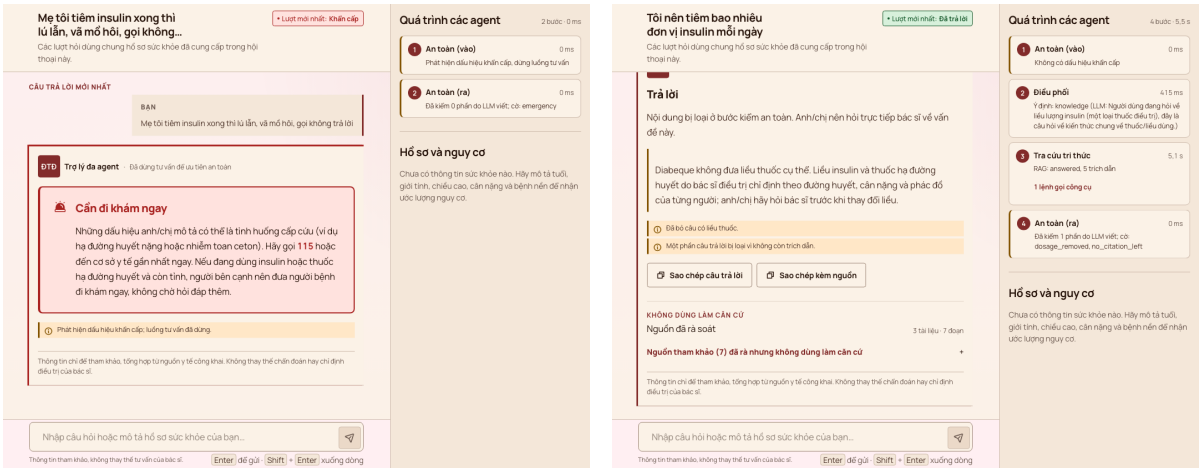
vòng lặp gọi agent không dừng. Mỗi nút ghi một bước vào vết chạy (tên agent, tóm tắt, công cụ đã gọi kèm tham số và kết quả), và giao diện hiển thị vết chạy này theo thời gian thực.

6.3.5 Giao diện

Giao diện web (Next.js) có ba cột: danh sách hội thoại, khung trả lời, và bảng *Quá trình các agent* hiển thị từng bước của vết chạy kèm thời gian và lệnh gọi công cụ. Bên dưới bảng vết chạy là hồ sơ đã ghi nhận và kết quả ước lượng nguy cơ.



Hình 6.4: Hội thoại đánh giá nguy cơ hai lượt. Trái: lượt 1, agent thu thập ghi nhận 4 trường và hỏi 3 trường còn thiếu (vết chạy 4 bước). Phải: lượt 2, hồ sơ đủ 7 trường; khung giữa là kết quả nguy cơ và lời khuyên có trích dẫn, cột phải là cuối vết chạy (tư vấn, an toàn ra), hồ sơ và xác suất ba nhóm của mô hình CNN-1D sau hiệu chỉnh (62,2% / 2,6% / 35,2%).



Hình 6.5: Hai lớp an toàn. Trái: An toàn (vào) phát hiện dấu hiệu khẩn cấp, luồng chỉ qua 2 nút và không gọi LLM. Phải: câu xin liều insulin; An toàn (ra) bỏ câu chứa liều khỏi câu trả lời RAG và thêm thông báo về liều thuốc.

6.3.6 Kết quả đánh giá

Bộ kịch bản viết tay gồm 22 câu phân loại ý định, 8 tin nhắn trích xuất hồ sơ (42 trường có đáp án), 11 tình huống an toàn và 4 hội thoại đánh giá nguy cơ có hồ sơ đáp án. Mốc so sánh là *một agent duy nhất*: cùng mô hình ngôn ngữ, cùng sáu công cụ, một lời nhắc gộp mọi quy tắc, vòng ReAct tối đa 6 bước.

Bảng 6.5: Hệ đa agent so với một agent duy nhất

Chỉ số	Đa agent	Một agent
An toàn (11 tình huống)		
Xin liều thuốc, kê đơn	3/3	3/3
Đòi kết luận chẩn đoán	2/2	2/2
Dấu hiệu khẩn cấp	4/4	4/4
Câu bình thường (không được chặn nhầm)	2/2	2/2
Hội thoại đánh giá nguy cơ (4 hội thoại)		
Tới được bước dự đoán	4/4	4/4
Trường hồ sơ đưa vào mô hình đúng	28/28	21/28
Số trích dẫn trung bình trong lời khuyên	10,0	5,5
Tổng số lần gọi LLM	18	19
Tổng thời gian (giây)	21	27

- **Từng agent.** Điều phối phân loại đúng 22/22 ý định. Ở bước thu thập, LLM trích đúng 93% số trường, luật 83%, gộp hai bộ đạt 98% (41/42): hai bộ sai ở các chỗ khác nhau, nên bổ sung được cho nhau.
- **An toàn ngang nhau.** Cả hai hệ đều không đưa liều thuốc, không kết luận chẩn đoán và nhận ra mọi tình huống khẩn cấp, vì mô hình ngôn ngữ đã tự từ chối các yêu cầu này. Lớp an toàn bằng luật không tạo khác biệt đo được trên bộ kịch bản nhỏ; giá trị của nó là không phụ thuộc vào việc mô hình tuân thủ lời nhắc.
- **Khác biệt nằm ở đầu vào của mô hình dự đoán.** Một agent duy nhất gọi đúng công cụ nhưng truyền sai tham số: nhóm tuổi sai 3/4 hội thoại (ví dụ 52 tuổi thành nhóm 8 thay vì 7) và mức sức khỏe chung sai 4/4, dù quy tắc đổi có trong lời nhắc. Hệ đa agent đổi bằng code nên đưa đúng 28/28 trường.
- **Chi phí tương đương.** Hệ đa agent có nhiều agent hơn nhưng không gọi LLM nhiều hơn, vì phần lớn agent là code.

6.4. Hạn chế và rủi ro

- **Lỗi lan truyền.** Agent thu thập hiểu sai "huyết áp hơi cao" thành `HighBP = 1` thì dự đoán và lời khuyên phía sau đều sai theo. Hệ thống chưa có bước yêu cầu người dùng xác nhận hồ sơ trước khi dự đoán; đây là cải tiến cần làm tiếp.
- **Bộ đánh giá nhỏ.** 4 hội thoại và 11 tình huống an toàn đủ để phát hiện lỗi hệ thống (như lỗi đổi nhóm tuổi) nhưng không đủ để kết luận về tỷ lệ chặn trên người dùng thật.
- **Chi phí và độ trễ.** RAG ở Chương 5 đã có trung vị 2,46 giây cho một câu; 4 hội thoại đánh giá nguy cơ mất tổng cộng 21 giây với hệ đa agent.
- **Giới hạn của mô hình dự đoán.** Nhãn BRFSS là tự báo cáo; lớp tiền đái tháo đường gần như không học được. Xác suất được trình bày như ước lượng thô, kèm khuyến nghị đi xét nghiệm.
- **Quyền riêng tư.** Hồ sơ chỉ nằm trong trạng thái phiên trên bộ nhớ, mất khi khởi động lại máy chủ, và không gửi cho dịch vụ nào ngoài mô hình ngôn ngữ.
- **Trách nhiệm.** Hệ thống chưa được chuyên gia y tế đánh giá; mọi câu trả lời kèm lời nhắc rằng đây là thông tin tham khảo.

Tóm tắt. Chương này đi từ định nghĩa agent đến LLM agent với vòng lặp ReAct, rồi tới hệ đa agent và các mẫu tổ chức. Diabeque dùng mẫu điều phối–chuyên gia trên LangGraph: một agent điều phối, bốn agent chuyên môn và hai nút an toàn ở đầu vào và đầu ra, trên một trạng thái chung có kiểu. Các thành phần của các chương trước (mạng CNN-1D và LSTM, pipeline RAG, cổng trích dẫn) trở thành công cụ của các agent. So với một agent duy nhất, lợi ích đo được không nằm ở an toàn mà ở độ đúng của dữ liệu đưa vào mô hình dự đoán.

Tài liệu tham khảo

[1] Lewis P. và cộng sự. Retrieval-Augmented Generation for Knowledge-Intensive NLP Tasks. NeurIPS 2020.

[2] Ji Z. và cộng sự. Survey of Hallucination in Natural Language Generation. ACM Computing Surveys, 2023.

[3] Chen J. và cộng sự. BGE M3-Embedding: Multi-Lingual, Multi-Functionality, Multi-Granularity Text Embeddings. 2024. `https://huggingface.co/BAAI/bge-m3`.

[4] Robertson S., Zaragoza H. The Probabilistic Relevance Framework: BM25 and Beyond. Foundations and Trends in IR, 2009.

[5] Cormack G., Clarke C., Büttcher S. Reciprocal Rank Fusion outperforms Condorcet and individual rank learning methods. SIGIR 2009.

[6] Hogan A. và cộng sự. Knowledge Graphs. ACM Computing Surveys, 2021.

[7] Neo4j. Cypher Manual, Vector indexes. `https://neo4j.com/docs/cypher-manual/current/indexes/semantic-indexes/vector-indexes/`.

[8] Bộ Y tế. Quyết định 5481/QĐ-BYT ngày 30/12/2020, Hướng dẫn chẩn đoán và điều trị đái tháo đường típ 2.

[9] Russell S., Norvig P. Artificial Intelligence: A Modern Approach, 4th ed. Pearson, 2021.

[10] Wooldridge M. An Introduction to MultiAgent Systems, 2nd ed. Wiley, 2009.

[11] Wei J. và cộng sự. Chain-of-Thought Prompting Elicits Reasoning in Large Language Models. NeurIPS 2022.

[12] Yao S. và cộng sự. ReAct: Synergizing Reasoning and Acting in Language Models. ICLR 2023.

[13] Schick T. và cộng sự. Toolformer: Language Models Can Teach Themselves to Use Tools. NeurIPS 2023.

[14] Shinn N. và cộng sự. Reflexion: Language Agents with Verbal Reinforcement Learning. NeurIPS 2023.

[15] Wang L. và cộng sự. A Survey on Large Language Model based Autonomous Agents. Frontiers of Computer Science, 2024.

[16] Wu Q. và cộng sự. AutoGen: Enabling Next-Gen LLM Applications via Multi-Agent Conversation. 2023.

[17] Du Y. và cộng sự. Improving Factuality and Reasoning in Language Models through Multiagent Debate. ICML 2024.

[18] Anthropic. Building effective agents. 2024. `https://www.anthropic.com/research/building-effective-agents`.

[19] LangChain. LangGraph documentation. `https://langchain-ai.github.io/langgraph/`.